Computer Vision Technique for Component Detection from SEM of FEOL Layer

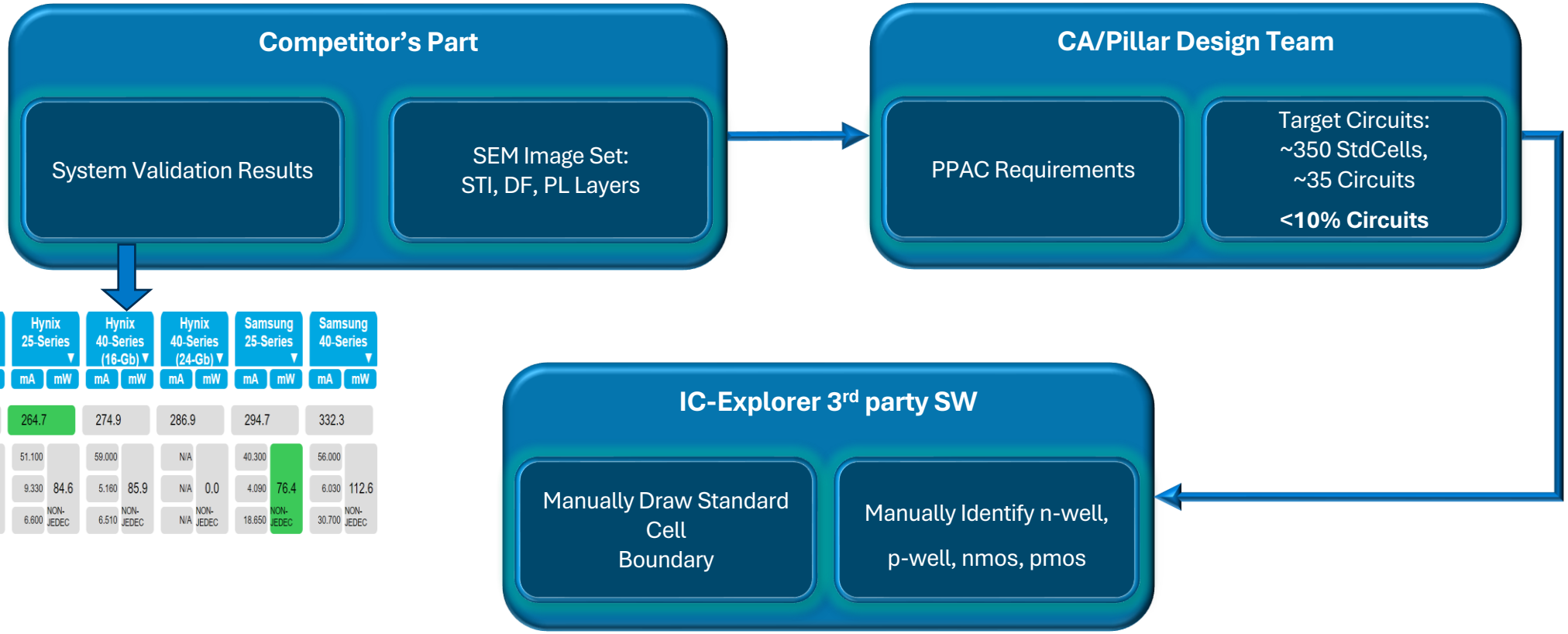
Jimit Shah, Prasun Agarwal, Santosh Paudel

Background and Limitations of Existing Solutions

- **Background:**
  - To understand the architecture and circuit trend of competitors, competitor analysis team captures Scanning Electron Microscope (SEM) images of different layers
  - The first step of understanding the circuits and eventually architecture of competitors is to understand shallow trench isolation area, n-well, p-well from Shallow Trench Isolation (STI), Diffusion (DF), and Polysilicon (PL) Layer's SEM image. This layers are also known as Front End of Line layers (FEOL)
  - Currently, we use a 3rd party software which imports SEM images. It allows to manually select these components by drawing bounding boxes around them i.e., nmos, pmos, diode, resistors are detected manually
  - After the components has been identified, we manually draw boundary where there is Complementary MOS structure which essentially makes StdCells the competitors are using i.e., Inverter, NAND, etc.
- **Limitation of Existing Solutions:**
  - Understanding the Raw SEM image to manually identify components and draw StdCell boundary in 3rd party software is a time-consuming process even for an experienced engineer
    - Manual detection of FEOL layers takes **~2.5-man months** for a given competitor reverse engineering
    - Due to limited resources in RE, we reverse engineer **~35 circuits** from each competitor's part is looked at, which is **<10% of circuits**
  - The decision if the component is individual nmos, pmos, StdCell structure, or resistor etc. can only be made with combination of STI, DF, PL layers, which adds to the **complexity** as the scanning through multiple layer is required to make the decision
  - The SEM images have lots of features to identify, as the target area of reverse engineering increases, the process becomes **prone to errors** and hence need **multiple iterations**
  - SEM image of a **complex circuit** like Error Correction Code (ECC) is impossible to reverse engineer with the current manual approach
  - The manual effort put on a given reverse engineering project cannot be reused in the next competitor's analysis due to lack of automation
  - Insight to competitor's design rules on FEOL layers is key to understand their technological advancements. Manual approaches scan through a very small area of competitors product, hence, do not get much insights on the design rules.
  - Along with this, with the manual approach it is hard to find if competitor has infringed Micron's IP on FEOL layers

Micron Highly Confidential

Background: What is being done today? (existing solutions)



	Y4CA RWID006		Y52K RWID011		Hynix 25-Series		Hynix 40-Series (16-Gb)		Hynix 40-Series (24-Gb)		Samsung 25-Series		Samsung 40-Series	
	mA	mW	mA	mW	mA	mW	mA	mW	mA	mW	mA	mW	mA	mW
Power Calculation Rev 1p98 (mW)	354.4		282.6		264.7		274.9		286.9		294.7		332.3	
Vdd	75.800		38.100		51.100		59.000		N/A		40.300		56.000	
Idd0 2 Vpp	5.320	142.0	3.880	92.6	9.330	84.6	5.160	85.9	N/A	0.0	4.090	76.4	6.030	112.6
Vddq	37.320	NON-JEDEC	34.960	NON-JEDEC	6.600	NON-JEDEC	6.510	NON-JEDEC	N/A	NON-JEDEC	18.650	NON-JEDEC	30.700	NON-JEDEC

Improvement Opportunity:

- ***< 10% Circuits Explored***
- ***Component Identification and drawing StdCell Boundary takes about ~2.5-man months per competitor's RE product***

PPAC: Power, Performance, Array Efficiency, Cost
RE: Reverse Engineering

Invention: Description

- The primary objective of this invention is to extract components like nmos, pmos, resistor, and detecting StdCell boundary based on Shallow Trench Isolation (STI), Diffusion (DF), and Polysilicon (PL) Layer's SEM image using in-house developed, and industry known deep learning algorithms
- This invention introduces an innovative approach that leverages classical image processing, OpenCV and Computer Vision techniques to address this problem, thus, streamlining the competitor analysis process. Proposed approach shifts time to competitor's architecture insight left by ~2.25 months.
- The methodology is as followed
  - SEM images from STI, DF, and PL layers are gathered
  - N-well and P-well are identified while using YOLO model on STI and DF layers
  - Using Convolutional Neural Network on PL layer, we identify non-dummy polysilicon gate structure
  - Based on N-well, P-well, and non-dummy gate structures, NMOS and PMOS transistors are identified
  - StdCell boundary is identification is a 3-fold approach:
    - For a corresponding NMOS, check which are the PMOS which have same center point.
    - Check if they have same width while including dummy structure to the width calculation
    - Choose the one which is nearest and make pair of NMOS and PMOS
- Broadly, we have divided the process into a six-fold approach and tested on different types of SEM Images

Invention: Component and StdCell Boundary Detection

Training and Testing (Per Layer)

Image Data Set from IC Explorer – ST, DF, PL Layers

Labeled Image with pwell, pmos, nmos

Convolutional Neural Network: YOLO, R-CNN

Feature Extraction: p-well, n-well, contacts

Inference (Per Layer)

< 80%

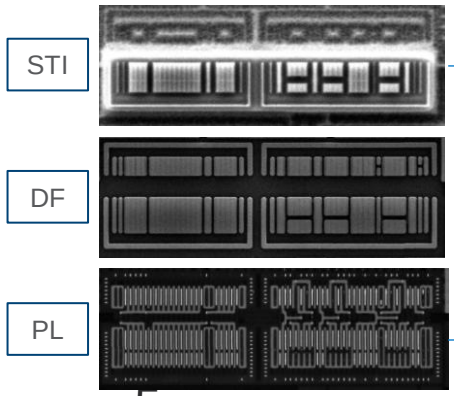
Update Parameters/ Algorithm

Accurate Working Models for Phase-1

> 80%

Accurate?

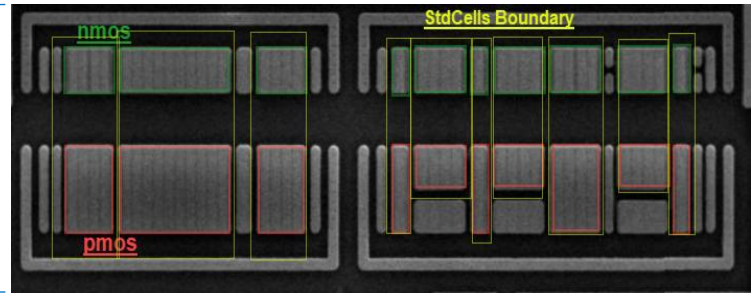
Classification: Pmos, Nmos, StdCell Boundary



Input

Phase-1

Output



Micron Highly Confidential

Methodology

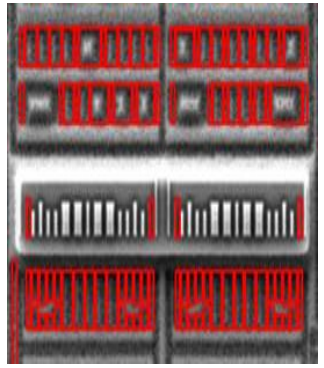
The Detection of Standard Cells from SEM images is broadly divided into 3 steps:

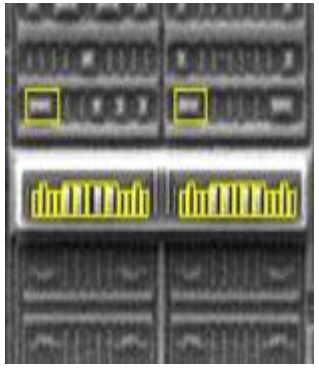
Step1: Detection of N-well and P-well from DF layer.

- Image Binarization
- Thresholding
- Contour Detection
- Contour Filtering

Step2: Classifying them into N-well and P-well using information from ST layer.

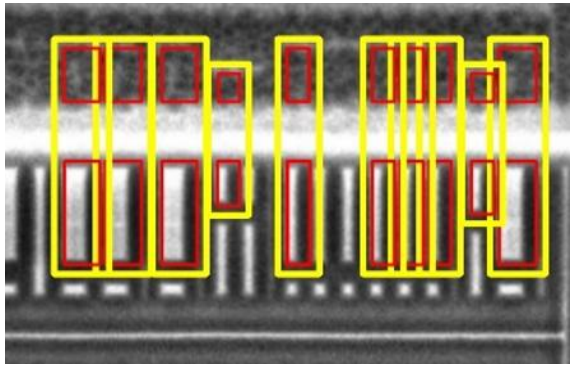
- Active contours (snakletes)
- Morphological Operations
- SAM (Segment Anything Model)
- Trench Viz





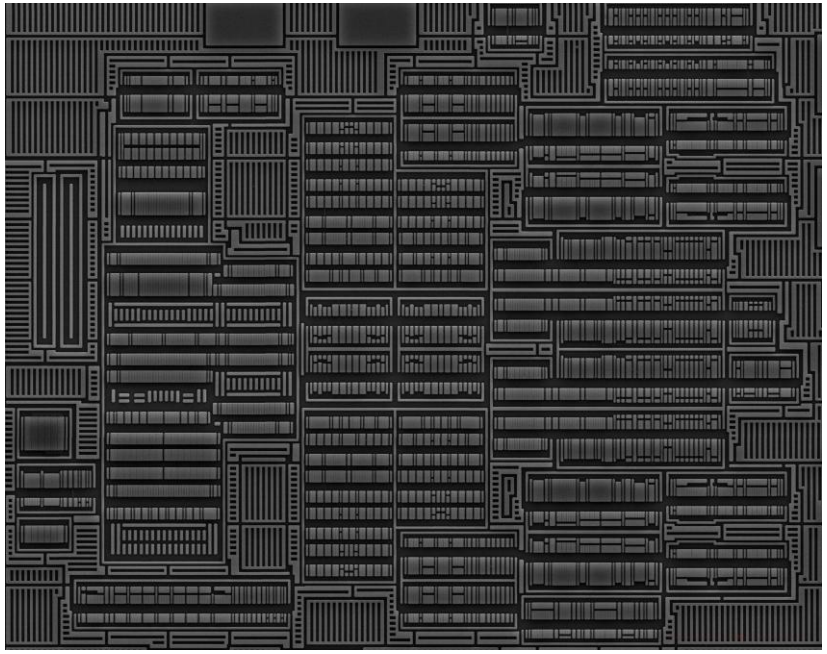
Step3: Put this information on PL layer for classification of Standard Cells.

- 3-fold approach
- Nearest distance and same centroid
- Should be Non-dummy

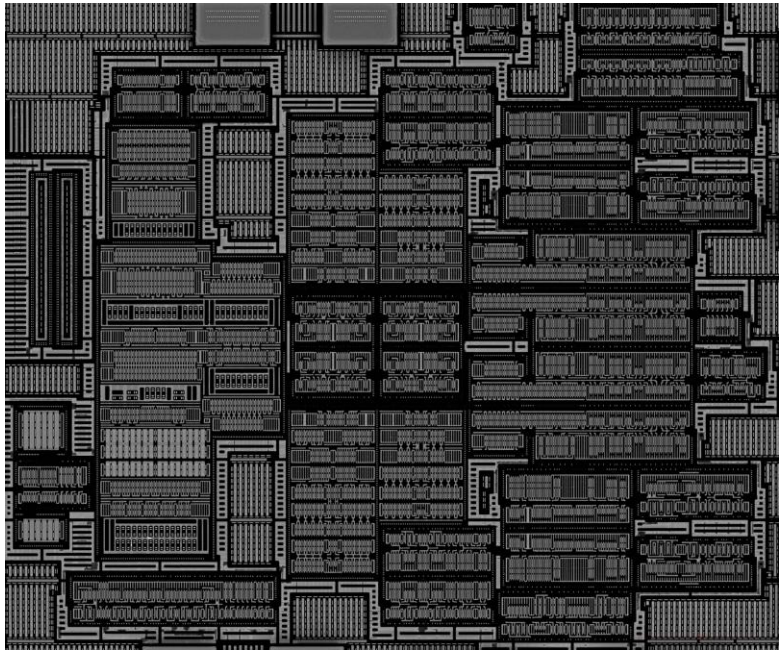


Input Images

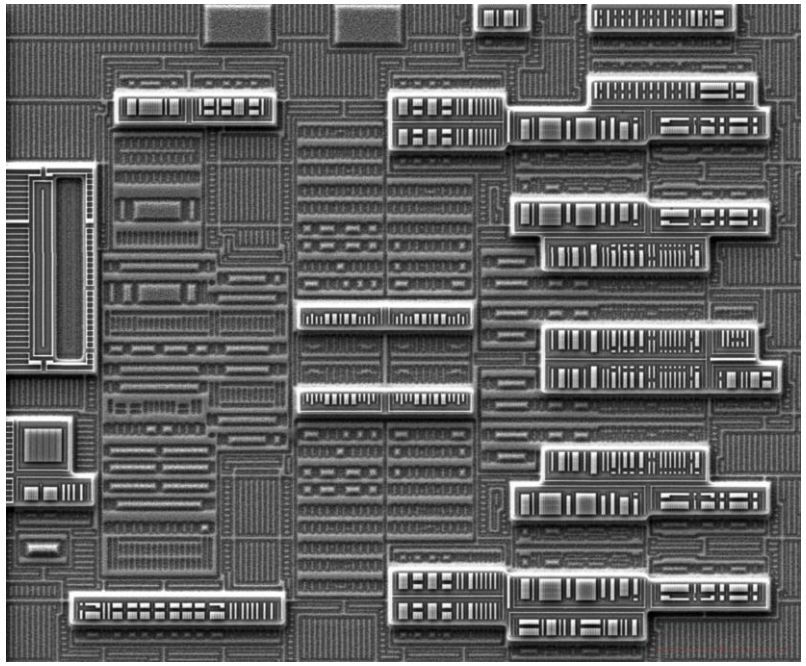
- Input Data is 3 layers of SEM FEOL Images



1. DF Layer (Diffusion layer)



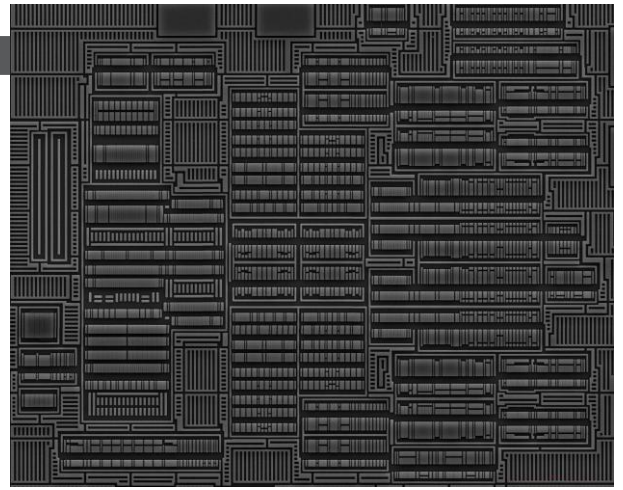
2. PL Layer (Poly-Silicon Layer)



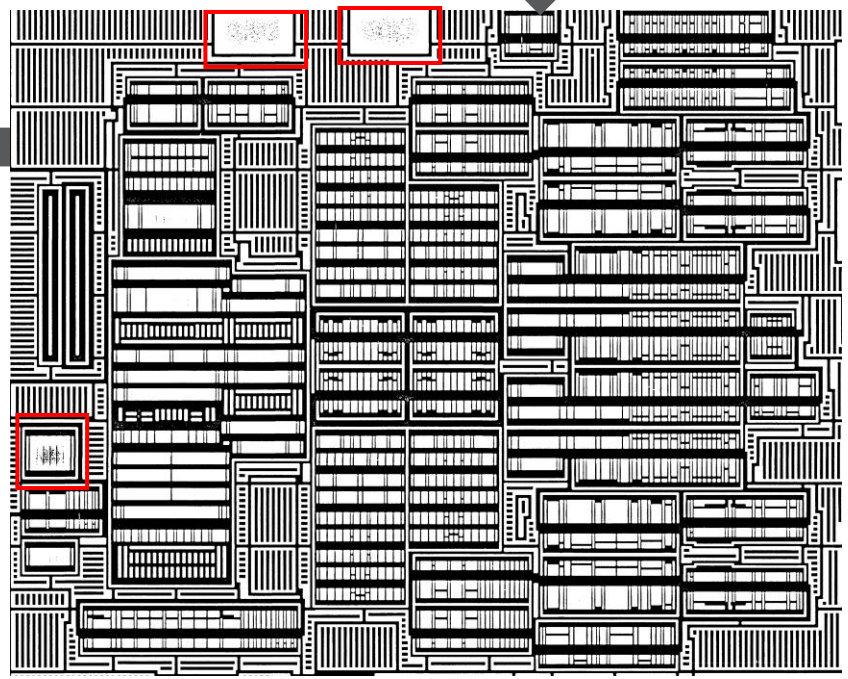
3. ST layer (Shallow Trench Layer)

Micron Highly Confidential

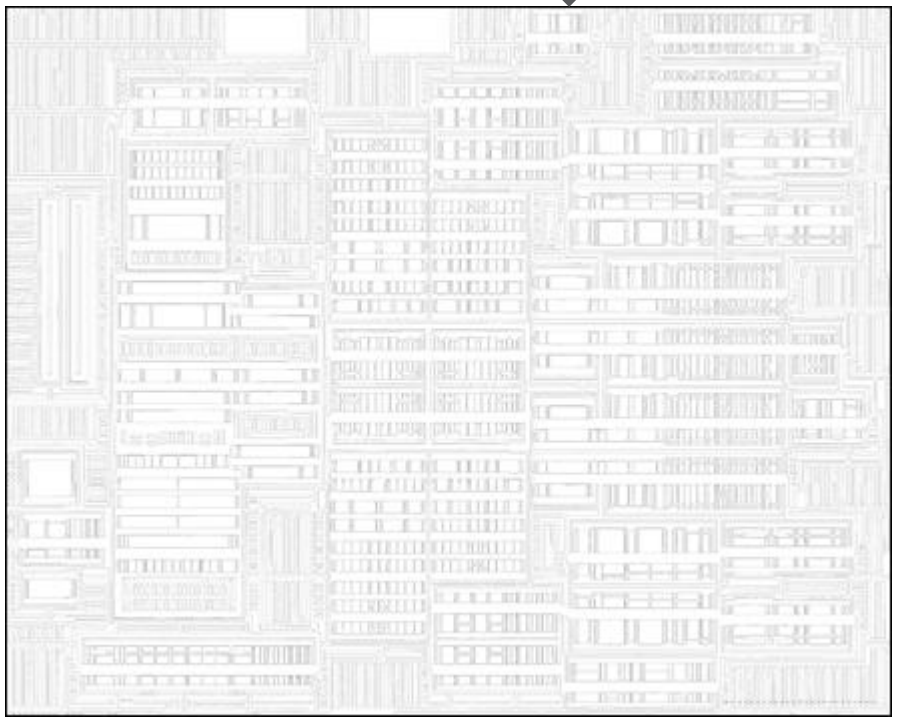
DF Layer Detection (Filtering Noise)



Step1: Given a DF layer, threshold the image to binarize the image (each pixel value is now 0 or 255),
Threshold : 80 to 120



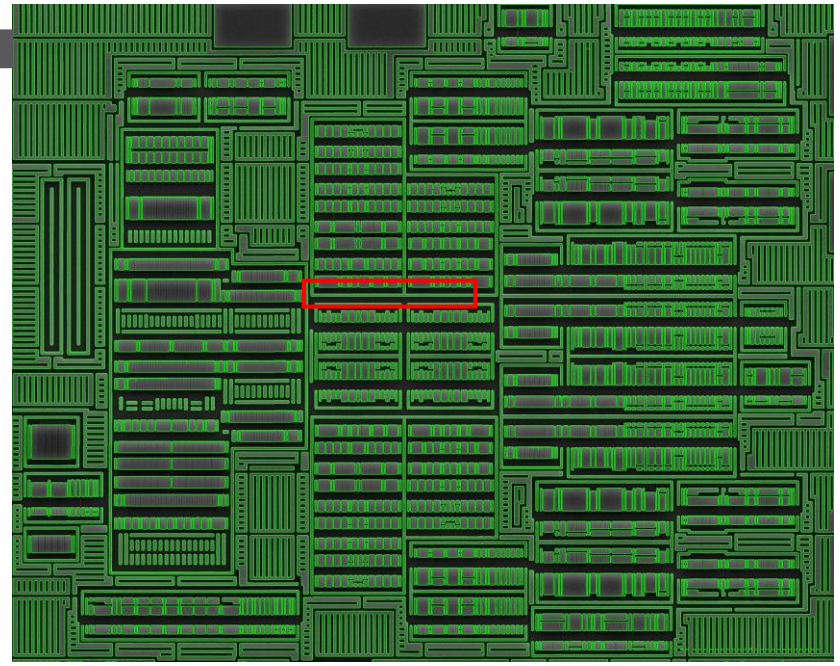
Step2: Use Gaussian Blurring followed by median filtering.
Parameter: Kernel Size 5*5



Step3: Detect Edges and Erosion.
Parameter: Canny Thresholds (50,100), Erosion kernel (5*5)

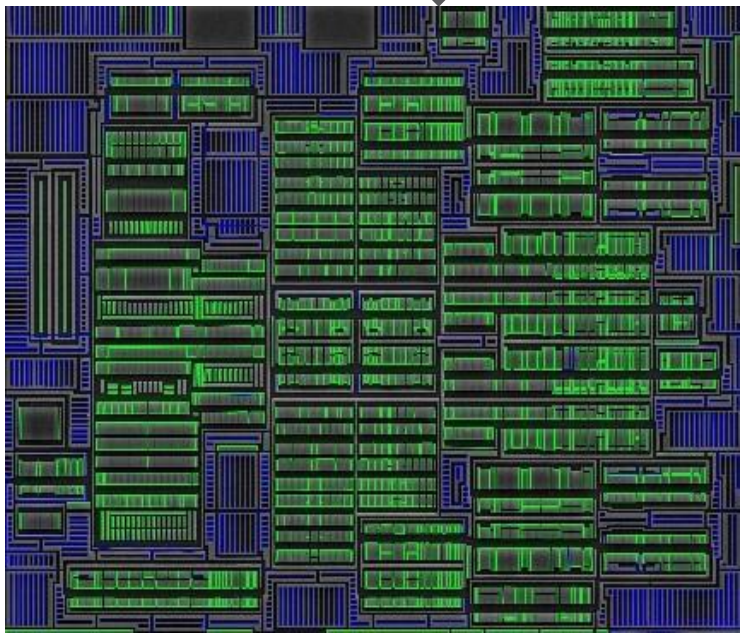
Micron Highly Confidential

Detection Potential N-well and P-wells



Step4: Detect all contours of all shapes.
Parameter: contourArea < 80000 and contourArea>50, center not in centroids

Step5: Remove noisy rectangles. Find theta with x-axis.
Parameter: Degree>87 or degree<3



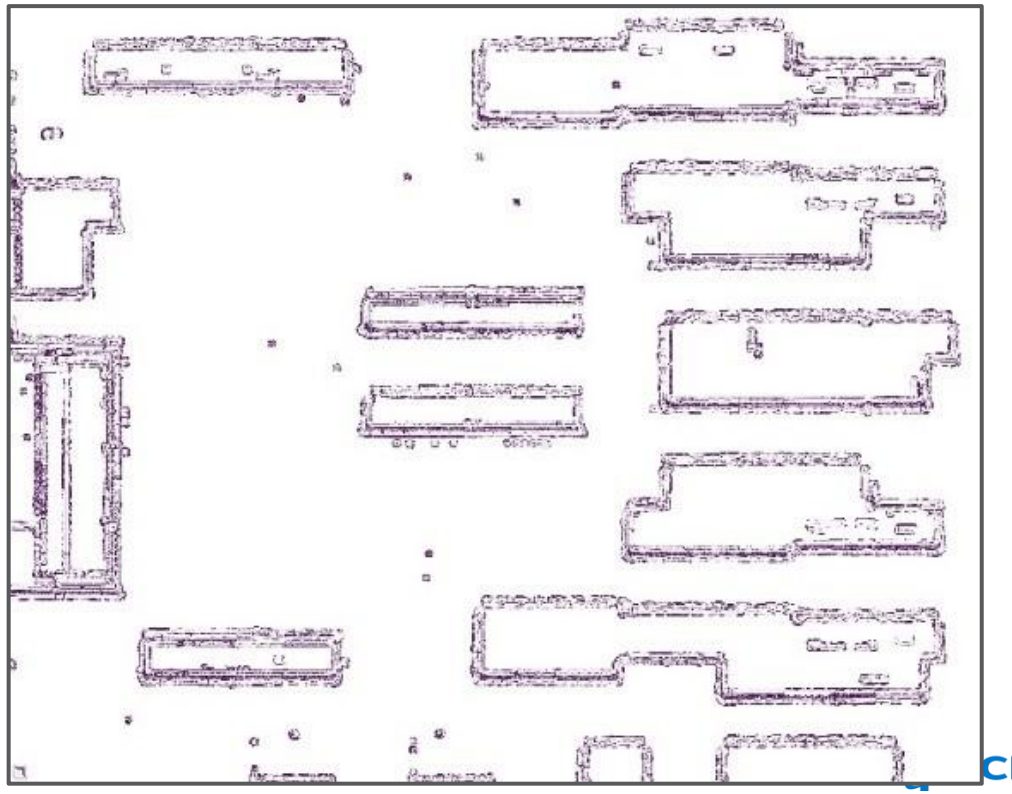
Step6: Detect all the Potential N-wells and Pwells
Parameter: Minimum sides =4,

Micron Highly Confidential

Shallow Trench Boxes Segmentation

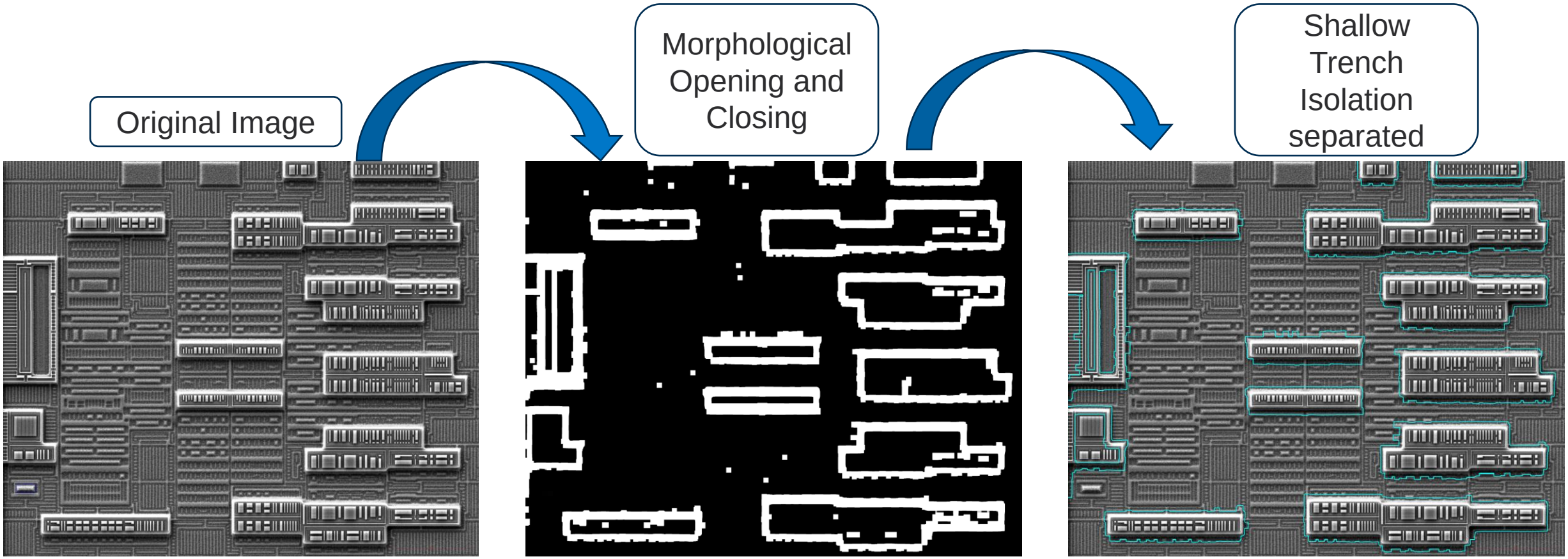
Morphological Transformations: This involves a process **of Opening and closing operations**. This means a dilation followed by erosion. Dilation is used to merge the broken contours and erosion is used to skeletonize the image back to normal. These boundaries are predicted after binarization of image.

- **Active Contours (Snakelets):** This method randomly expands in x and y direction taking care of strength of expansion and length of expansion to get the contours of our interest. This is slow and computationally expensive as many combinations are tried in all directions to get the strong contours.



Micron Highly Confidential

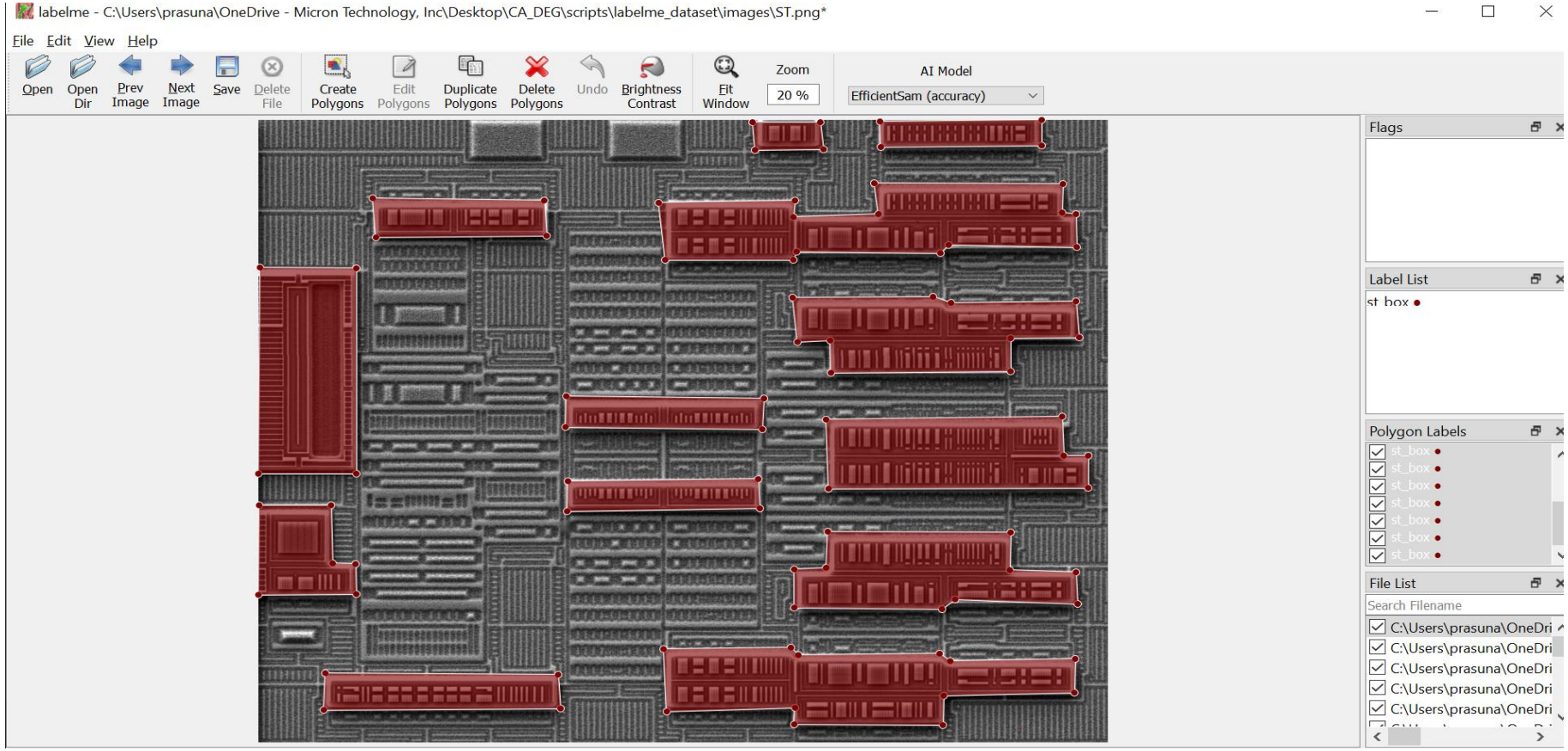
Erosion/ Dilation



- Not scalable
- Does not generalize well
- Too many user specific parameters to be adjusted

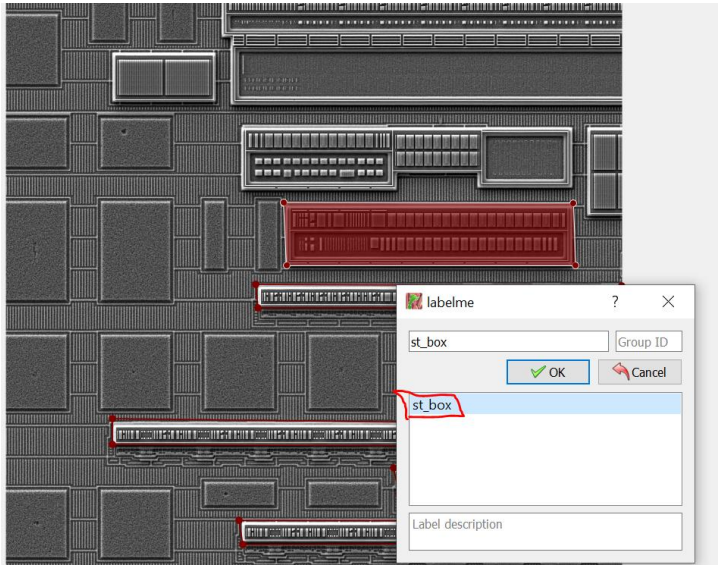
Creating Dataset

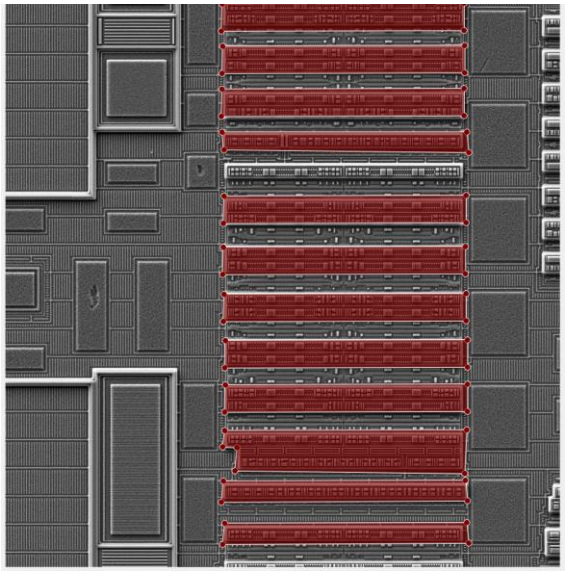
- Image Polygonal Annotation with Python
- Labelme is the tool used to mark the manual annotations to the images. (https://github.com/labelmeai/labelme)
- Labelme is a graphical image annotation tool.

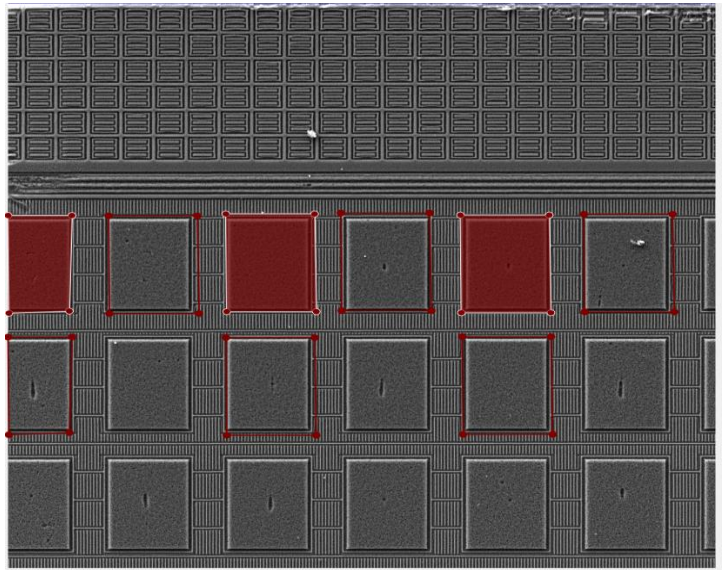


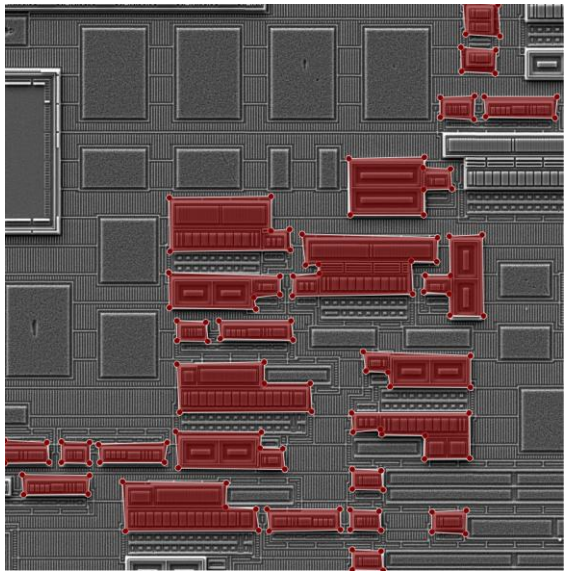
Other Image Annotations

- Around 30+ images manually annotated.
- Each Image had average of around 5-6 masks.
- Deliberate noise was introduced in the training data to make the model learn better.
- Single class classification label (st_box)



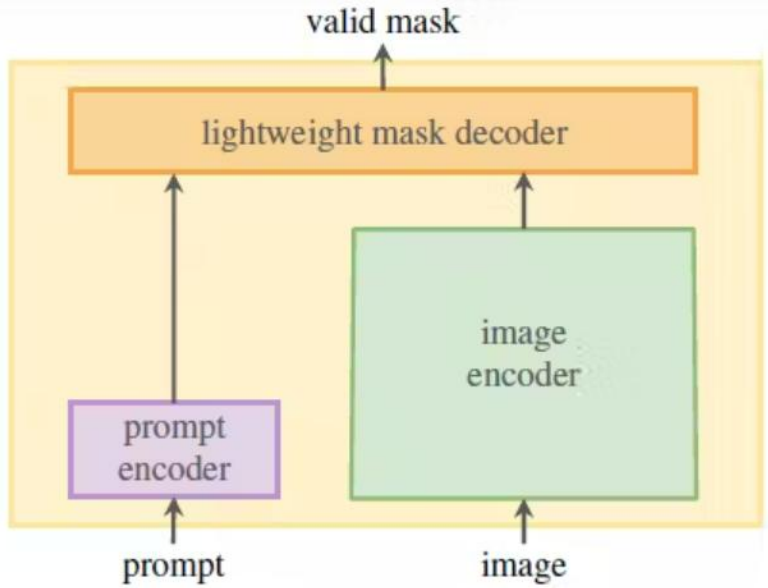






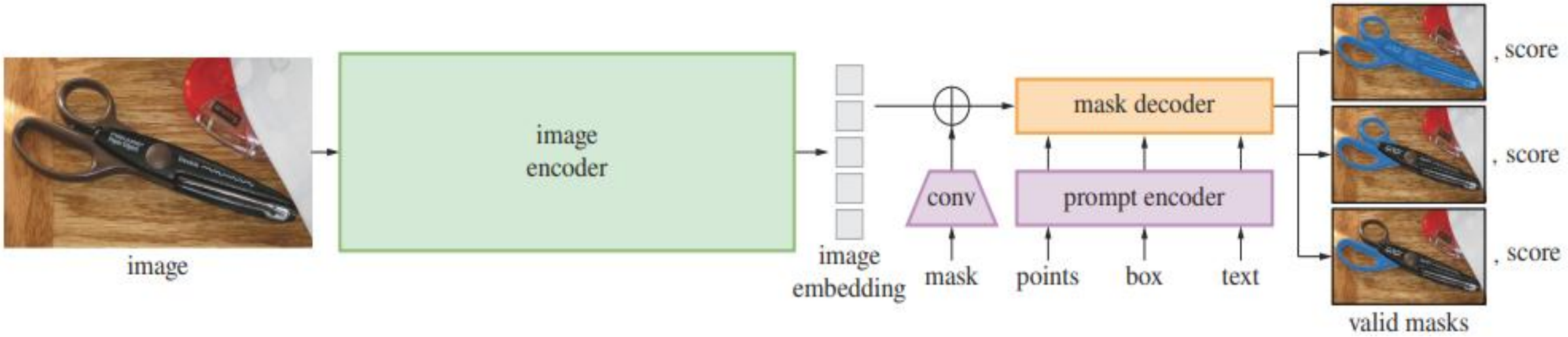
About SAM (Segment Anything Model)

- Segment Anything (SAM) is an image segmentation model developed by Meta AI.
- SAM was released in April 2023.
- The Segment Anything Model employs
  - A powerful image encoder,
  - A prompt encoder, and
  - A lightweight mask decoder.
- **Zero-Shot Performance:** SAM displays outstanding zero-shot performance across various segmentation tasks
- The Segment Anything Model offers a powerful and versatile solution for object segmentation in images, enabling you to enhance your datasets with segmentation masks.
- With its fast-processing speed and various modes of inference, SAM is a valuable tool for computer vision applications.

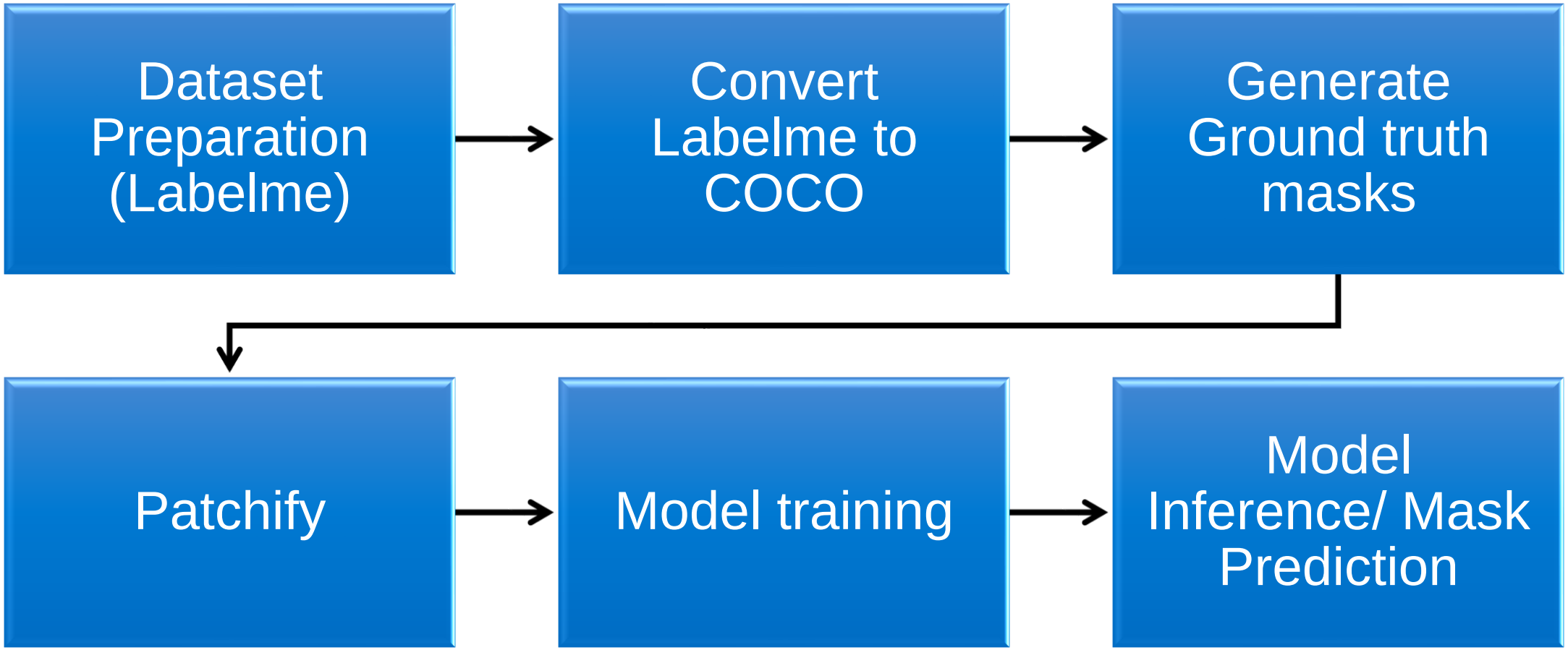


SAM Model Architecture

- SAM's architecture comprises three components that work together to return a valid segmentation mask:
  - An image encoder: To generate one-time image embeddings. It uses a masked auto-encoder, MAE, pre-trained Vision Transformer to generate one-time image embeddings.
  - A prompt encoder: It encodes/ embeds background points, masks, bounding boxes, or texts into an embedding vector in real-time.
  - A lightweight mask decoder: It combines the embeddings from the prompt and image encoders. A lightweight mask decoder predicts the segmentation masks. The mask decoder efficiently maps the image and prompt embeddings to generate segmentation masks.



SAM Model Process Flowchart



SAM Model Data Preparation

- Converted the manually annotated images into Ground truth masks.
- These ground truth masks are used for model training.
- Converting Labelme Annotations to COCO annotations for to be used by SAM model
- Generate Binary masks of Image

```
1  import labelme2coco
```

```
1  set directory that contains labelme annotations and image
2  labelme_folder = r"C:\Users\prasuna\OneDrive - Micron Te
3
4  set path for coco json to be saved
5  save_json_path = r"C:\Users\prasuna\OneDrive - Micron Te
6
7  convert labelme annotations to coco
8  labelme2coco.convert(labelme_folder, save_json_path)
```

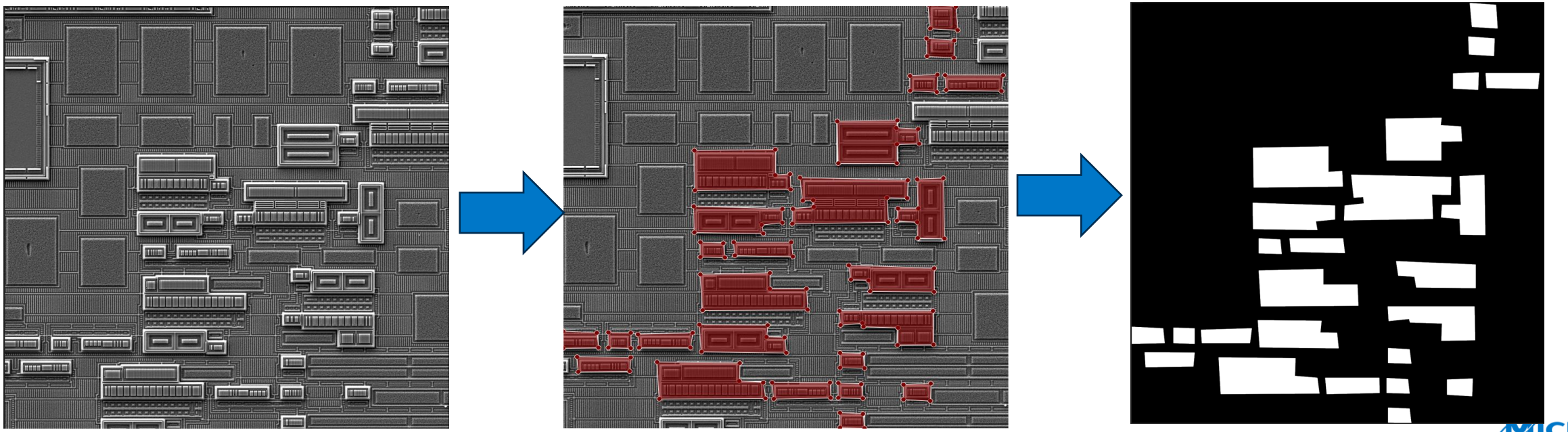
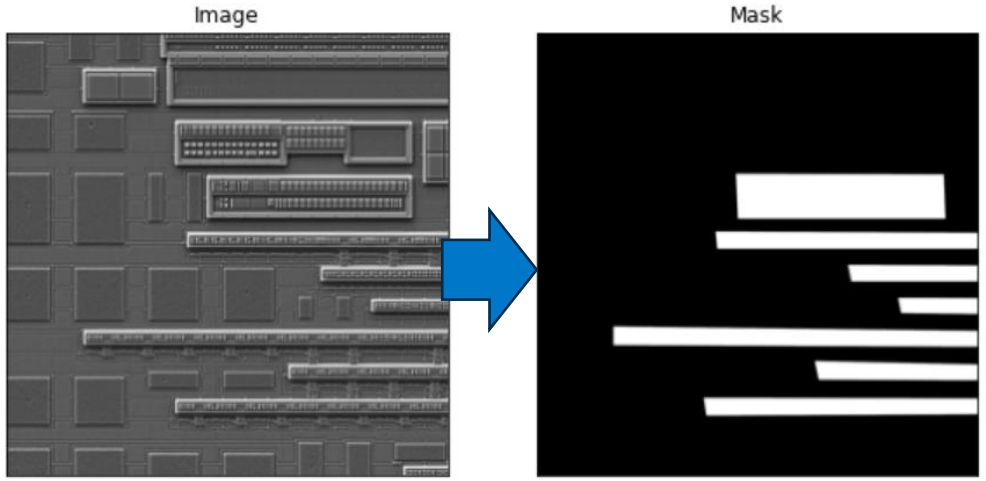
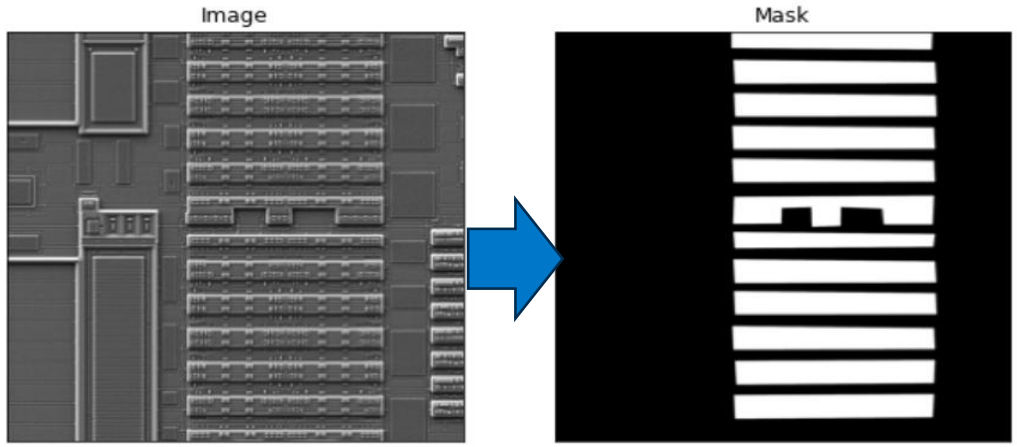
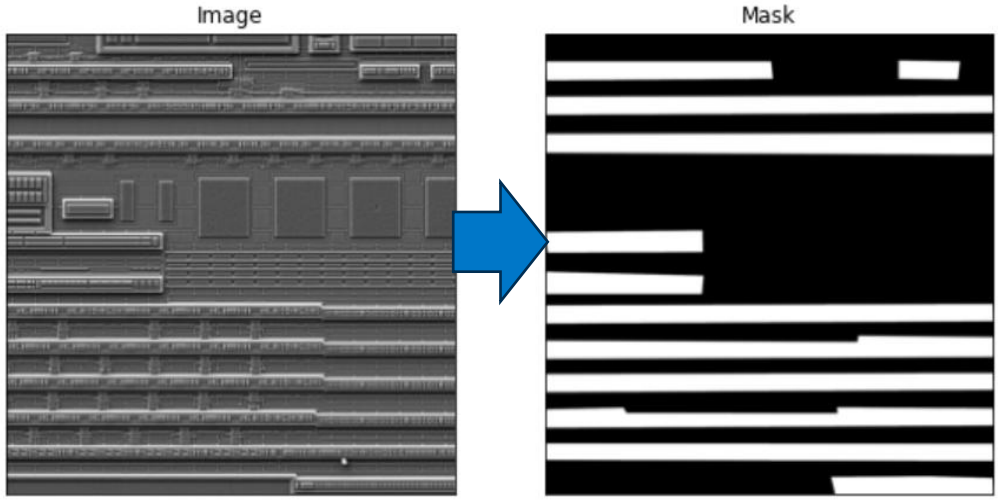
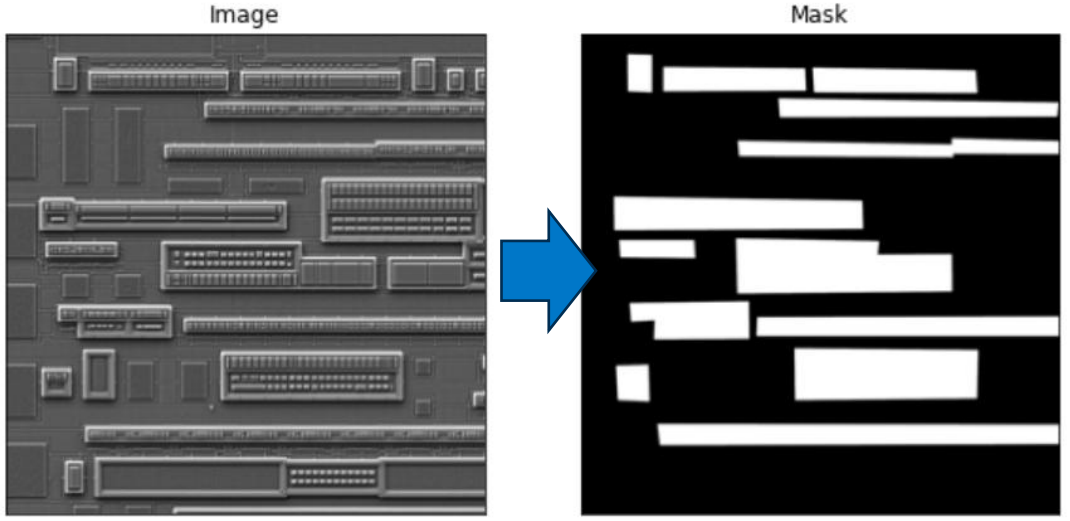


Image Mask Generation Examples



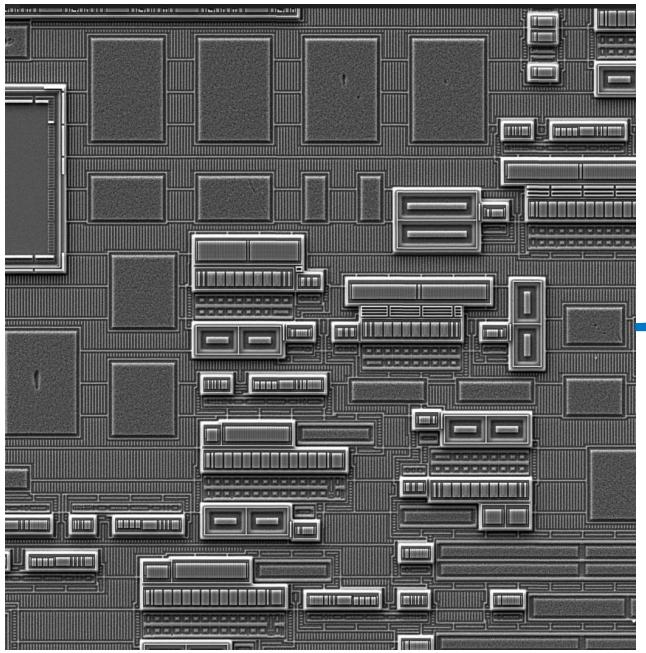






Patchify

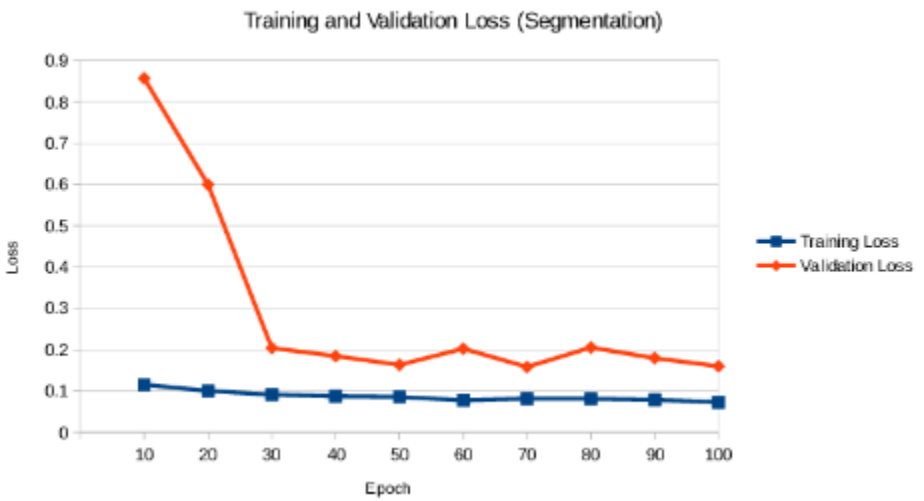
- Dividing the big image into small size of 256*256 images in step size of 256.
- Using Patchify approach the generate the patches

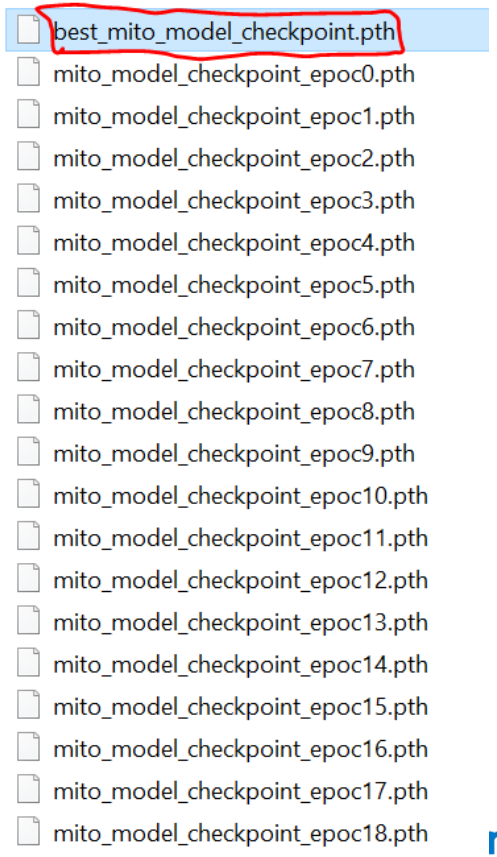


Model Training

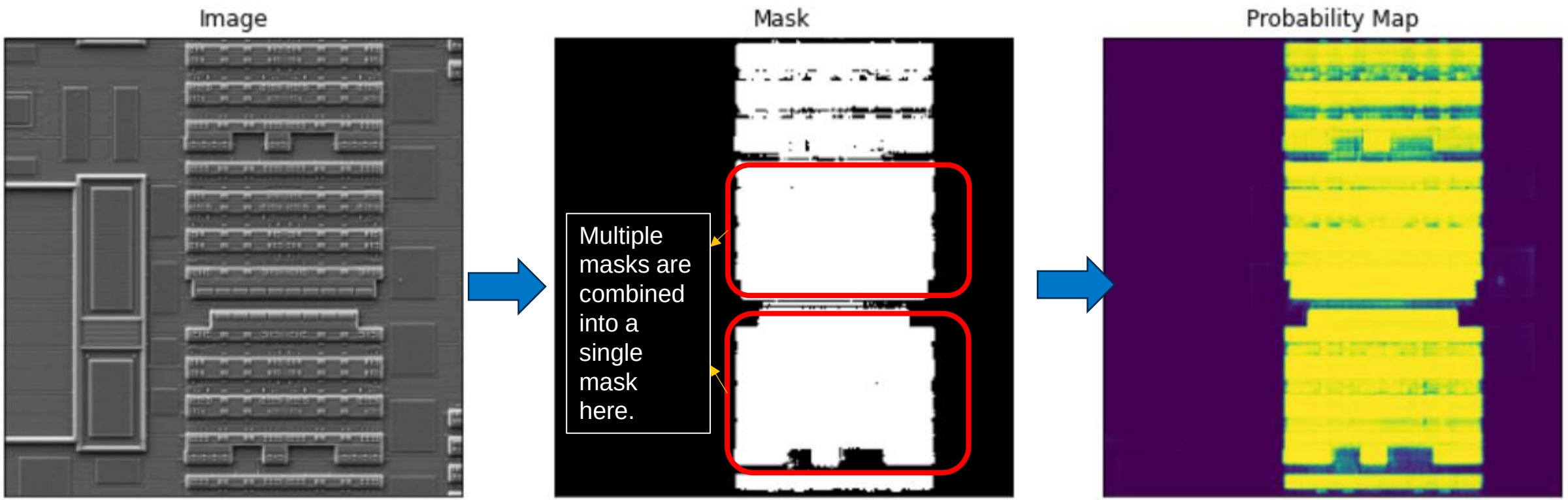
- Custom trained SAM model on the Labelled dataset.
- Epochs: 100
- Used Monai Loss function for segmentation: Compute average Dice loss between two tensors. It can support both multi-classes and multi-labels tasks.
- Optimizer: ADAM
- Reduce Mean loss
- Accuracy: 59%

```
100%|██████████| 15/15 [07:52<00:00, 31.47s/it]
EPOCH: 0
Mean loss: -180.6070381641388
100%|██████████| 15/15 [08:28<00:00, 33.92s/it]
EPOCH: 1
Mean loss: -539.315717569987
100%|██████████| 15/15 [08:19<00:00, 33.32s/it]
EPOCH: 2
Mean loss: -1894.239697265625
100%|██████████| 15/15 [07:31<00:00, 30.07s/it]
EPOCH: 3
Mean loss: -5732.716097005208
```





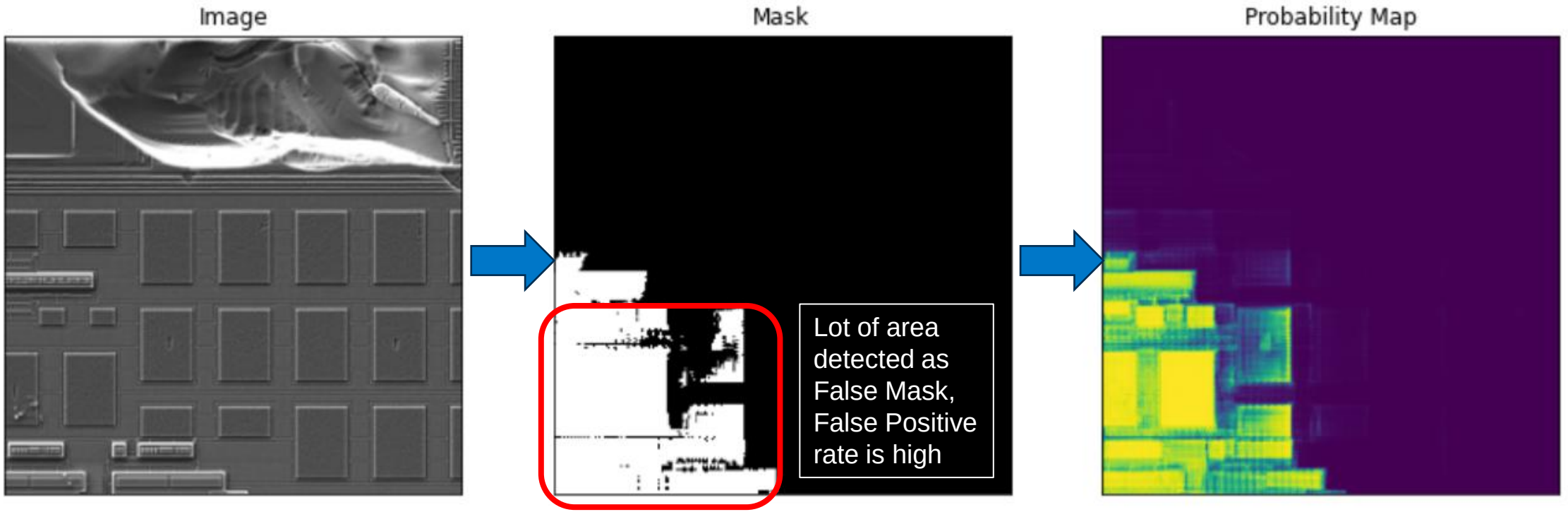
SAM Prediction results (Sample1)



Observations:

- **Time for Prediction:** 0.2 sec
- **Segmentation Accuracy:** 55%
- Does not generalize well, combines multiple masks into a single mask.
- Low probability confidence of prediction.

SAM Prediction results (Sample2)

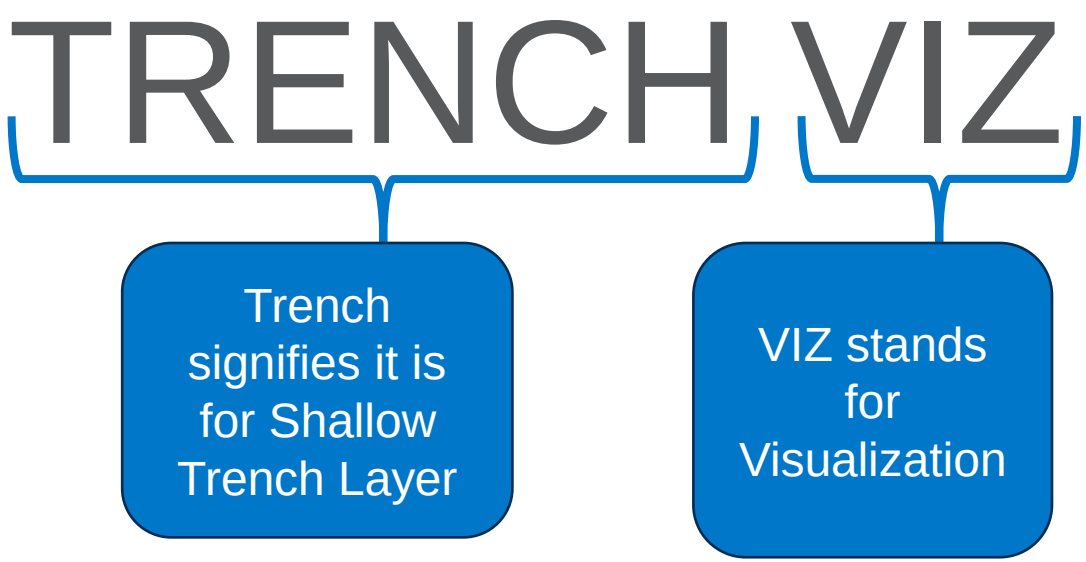


Observations:

- **Time for Prediction:** 0.2 sec
- **Segmentation Accuracy:** 35%
- Does not generalize well, combines multiple masks into a single mask.
- Low probability confidence of prediction.
- Lot of False Positive pixels been segmented, higher FP rate.

TRENCH VIZ

- It is an in-house built and trained Deep learning model
- It is based on the architecture on YoloV8
- It is trained from scratch on the custom dataset which is self annotated.



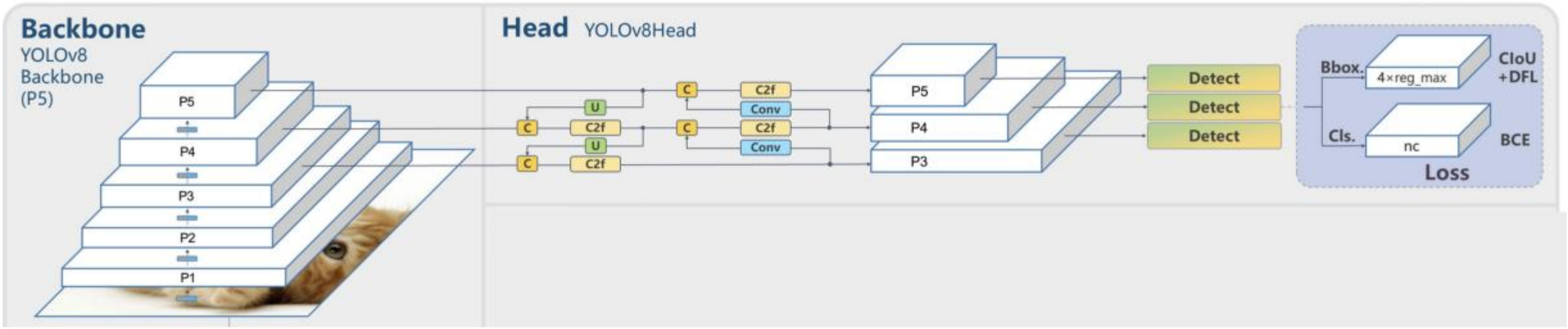
About Trench Viz

- It is a state-of-the-art **deep learning model** for **real-time object detection** in computer vision applications.
- Its advanced architecture and algorithms enable accurate and efficient **object detection**.
- Image segmentation goes beyond object detection by identifying the exact shape and boundaries of objects within an image. It provides pixel-level information about each object, enabling more detailed analysis and understanding of the image content. While image segmentation can be computationally expensive, **Trench Viz** integrates this method into its neural network architecture, allowing for efficient and accurate object segmentation.
- **Trench Viz** does the following three step process:

Method	Approach	Use Case
Classification	Assigning class labels to an entire image	Identifying general content or context of an image
Object Detection	Identifying and locating multiple objects within an image	Autonomous driving, robotics, video surveillance
Image Segmentation	Identifying exact shapes and boundaries of objects	Detailed image analysis and understanding

YOLOv8 Base Model Architecture

- YOLOv8 is the latest version of the YOLO object detection model. This latest version has the same architecture as its predecessors, but it introduces numerous improvements compared to the earlier versions of YOLO.
- It utilizes both Feature Pyramid Network (FPN) and Path Aggregation Network (PAN)
- The FPN works by gradually reducing the spatial resolution of the input image while increasing the number of feature channels. This results in the creation of feature maps that are capable of detecting objects at different scales and resolutions.
- The PAN architecture, on the other hand, aggregates features from different levels of the network through skip connections. By doing so, the network can better capture features at multiple scales and resolutions.



Trench Viz Architecture

About Trench Viz

- Few layers in the backbone and head were dropped in the New custom architecture.
- These layers were needed for scale adjustments.

Advantage:

- Time and Cost Reduction
- Because of Lightweight this Model can be used on Edge Devices
- Can be used on all the FEOL SEM images

YoloV8 Architecture

```yaml
nc: 80  # number of classes
scales: # model compound scaling constants, i.e. 'model=yolov8n-seg.yaml'
  # [depth, width, max_channels]
  n: [0.33, 0.25, 1024]
  s: [0.33, 0.50, 1024]
  m: [0.67, 0.75, 768]
  l: [1.00, 1.00, 512]
  x: [1.00, 1.25, 512]

# YOLOv8.0n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]]  # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]  # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]  # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]]  # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  - [-1, 1, Conv, [1024, 3, 2]]  # 7-P5/32
  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, SPPF, [1024, 5]]  # 9

# YOLOv8.0n head
head:
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 6], 1, Concat, [1]]  # cat backbone P4
  - [-1, 3, C2f, [512]]  # 12

  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]]  # cat backbone P3
  - [-1, 3, C2f, [256]]  # 15 (P3/8-small)

  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 12], 1, Concat, [1]]  # cat head P4
  - [-1, 3, C2f, [512]]  # 18 (P4/16-medium)

  - [-1, 1, Conv, [512, 3, 2]]
  - [[-1, 9], 1, Concat, [1]]  # cat head P5
  - [-1, 3, C2f, [1024]]  # 21 (P5/32-large)

  - [[15, 18, 21], 1, Segment, [nc, 32, 256]]  # Segment(P3, P4, P5)
```

Trench Viz Architecture

```yaml
nc: 1  # number of classes
scales: # model compound scaling constants, i.e. 'model=yolov8n-seg.y
  # [depth, width, max_channels]
  n: [0.33, 0.25, 256]
  s: [0.33, 0.50, 1024]
  m: [0.67, 0.75, 768]
  l: [1.00, 1.00, 512]
  x: [1.00, 1.25, 512]

# YOLOv8.0n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]]  # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]]  # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]]  # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]]  # 5-P4/16
  - [-1, 6, C2f, [512, True]]
#  - [-1, 1, Conv, [1024, 3, 2]]  # 7-P5/32
#  - [-1, 3, C2f, [1024, True]]
  - [-1, 1, SPPF, [1024, 5]]  # 9

# YOLOv8.0n head
head:
  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 6], 1, Concat, [1]]  # cat backbone P4
  - [-1, 3, C2f, [512]]  # 12

  - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [[-1, 4], 1, Concat, [1]]  # cat backbone P3
  - [-1, 3, C2f, [256]]  # 15 (P3/8-small)

  - [-1, 1, Conv, [256, 3, 2]]
  - [[-1, 12], 1, Concat, [1]]  # cat head P4
  - [-1, 3, C2f, [512]]  # 18 (P4/16-medium)

#  - [-1, 1, Conv, [512, 3, 2]]
#  - [[-1, 9], 1, Concat, [1]]  # cat head P5
#  - [-1, 3, C2f, [1024]]  # 21 (P5/32-large)

  - [[15, 18, 21], 1, Segment, [nc, 32, 256]]  # Segment(P3, P4, P5)
```

Micron Highly Confidential

Parameter Reductions

- The number of parameters reduced **from 11 Million to 3 millions (72% reduction).**
- Good accuracy results were obtained using the smaller size model.

YoloV8 Parameters

	from	n	params	module	arguments
0	-1	1	928	ultralytics.nn.modules.Conv	[3, 32, 3, 2]
1	-1	1	18560	ultralytics.nn.modules.Conv	[32, 64, 3, 2]
2	-1	1	29056	ultralytics.nn.modules.C2f	[64, 64, 1, True]
3	-1	1	73984	ultralytics.nn.modules.Conv	[64, 128, 3, 2]
4	-1	2	197632	ultralytics.nn.modules.C2f	[128, 128, 2, True]
5	-1	1	295424	ultralytics.nn.modules.Conv	[128, 256, 3, 2]
6	-1	2	788480	ultralytics.nn.modules.C2f	[256, 256, 2, True]
7	-1	1	1180672	ultralytics.nn.modules.Conv	[256, 512, 3, 2]
8	-1	1	1838080	ultralytics.nn.modules.C2f	[512, 512, 1, True]
9	-1	1	656896	ultralytics.nn.modules.SPPF	[512, 512, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.Concat	[1]
12	-1	1	591360	ultralytics.nn.modules.C2f	[768, 256, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.Concat	[1]
15	-1	1	148224	ultralytics.nn.modules.C2f	[384, 128, 1]
16	-1	1	147712	ultralytics.nn.modules.Conv	[128, 128, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.Concat	[1]
18	-1	1	493056	ultralytics.nn.modules.C2f	[384, 256, 1]
19	-1	1	590336	ultralytics.nn.modules.Conv	[256, 256, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.Concat	[1]
21	-1	1	1969152	ultralytics.nn.modules.C2f	[768, 512, 1]
22	[15, 18, 21]	1	2775392	ultralytics.nn.modules.Segment	[14, 32, 128, [128,

YOLOv8s-seg summary: 261 layers, 11795514 parameters, 11795498 gradients, 42.7 GFLOPs

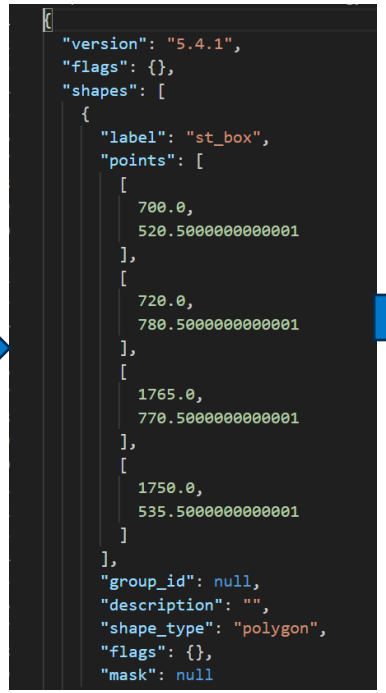
Trench Viz Parameters

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.Conv	[16, 32, 3, 2]
2	-1	1	7360	ultralytics.nn.modules.C2f	[32, 32, 1, True]
3	-1	1	18560	ultralytics.nn.modules.Conv	[32, 64, 3, 2]
4	-1	2	49664	ultralytics.nn.modules.C2f	[64, 64, 2, True]
5	-1	1	73984	ultralytics.nn.modules.Conv	[64, 128, 3, 2]
6	-1	2	197632	ultralytics.nn.modules.C2f	[128, 128, 2, True]
7	-1	1	295424	ultralytics.nn.modules.Conv	[128, 256, 3, 2]
8	-1	1	460288	ultralytics.nn.modules.C2f	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.SPPF	[256, 256, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.Concat	[1]
12	-1	1	148224	ultralytics.nn.modules.C2f	[384, 128, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.Concat	[1]
15	-1	1	37248	ultralytics.nn.modules.C2f	[192, 64, 1]
16	-1	1	36992	ultralytics.nn.modules.Conv	[64, 64, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.Concat	[1]
18	-1	1	123648	ultralytics.nn.modules.C2f	[192, 128, 1]
19	-1	1	147712	ultralytics.nn.modules.Conv	[128, 128, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.Concat	[1]
21	-1	1	493056	ultralytics.nn.modules.C2f	[384, 256, 1]
22	[15, 18, 21]	1	1004875	ultralytics.nn.modules.Segment	[1, 32, 64, [64, 128, 256]]

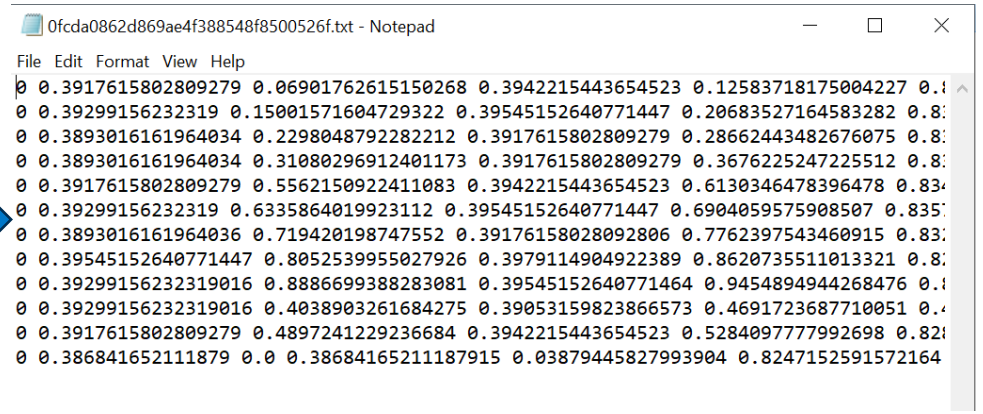
YOLOv8n-seg summary: 261 layers, 3263811 parameters, 3263795 gradients, 12.1 GFLOPs

Labelme2Yolo

- Yolo accepts data in txt Yolo format
- Converted the Labelme annotations (Json format) to Yolo annotations (txt format)



```
{
  "version": "5.4.1",
  "flags": {},
  "shapes": [
    {
      "label": "st_box",
      "points": [
        [
          700.0,
          520.5000000000001
        ],
        [
          720.0,
          780.5000000000001
        ],
        [
          1765.0,
          770.5000000000001
        ],
        [
          1750.0,
          535.5000000000001
        ]
      ],
      "group_id": null,
      "description": "",
      "shape_type": "polygon",
      "flags": {},
      "mask": null
```



0fcda0862d869ae4f388548f8500526f.txt - Notepad

File Edit Format View Help

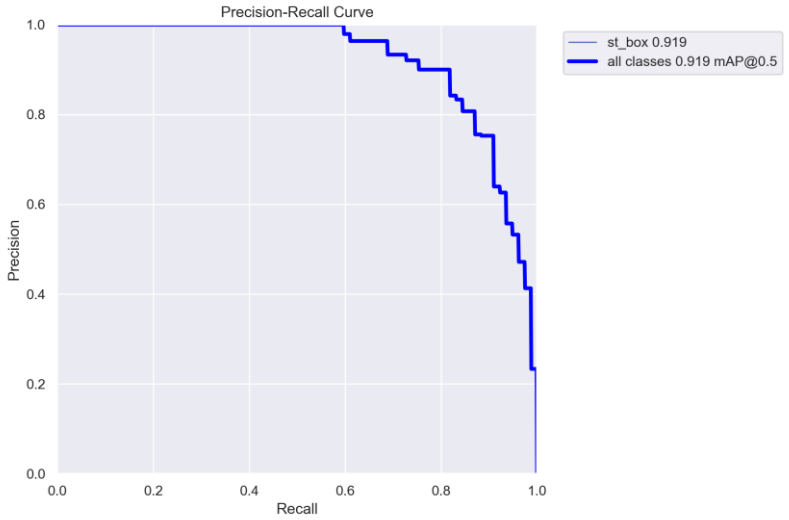
```
0 0.3917615802809279 0.06901762615150268 0.3942215443654523 0.12583718175004227 0.8
0 0.39299156232319 0.15001571604729322 0.39545152640771447 0.20683527164583282 0.83
0 0.3893016161964034 0.2298048792282212 0.3917615802809279 0.28662443482676075 0.83
0 0.3893016161964034 0.31080296912401173 0.3917615802809279 0.3676225247225512 0.83
0 0.3917615802809279 0.5562150922411083 0.3942215443654523 0.6130346478396478 0.834
0 0.39299156232319 0.6335864019923112 0.39545152640771447 0.6904059575908507 0.835
0 0.3893016161964036 0.719420198747552 0.39176158028092806 0.7762397543460915 0.832
0 0.39545152640771447 0.8052539955027926 0.3979114904922389 0.8620735511013321 0.82
0 0.39299156232319016 0.8886699388283081 0.39545152640771464 0.9454894944268476 0.8
0 0.39299156232319016 0.4038903261684275 0.39053159823866573 0.4691723687710051 0.4
0 0.3917615802809279 0.4897241229236684 0.3942215443654523 0.5284097777992698 0.828
0 0.386841652111879 0.0 0.38684165211187915 0.03879445827993904 0.8247152591572164
```

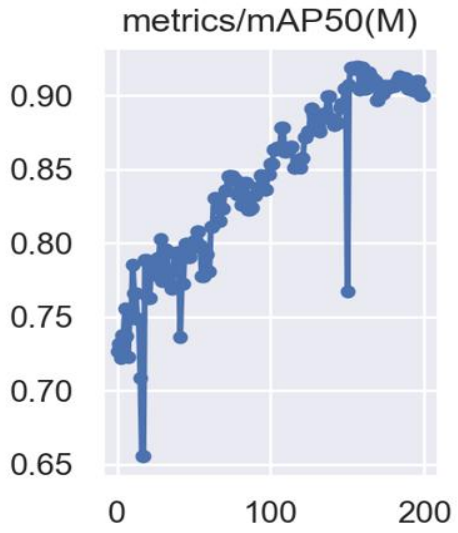
Trench Viz Training

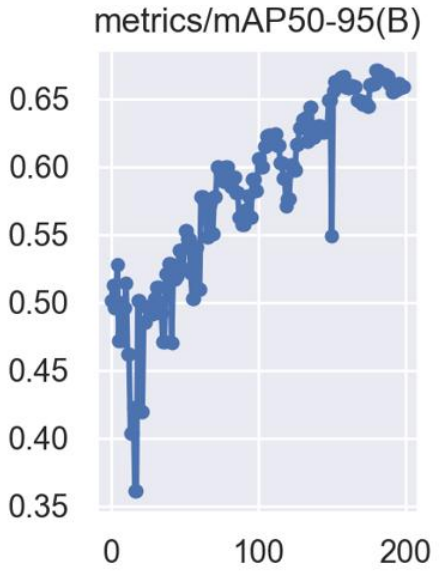
- Trained on 200 epochs
- Train dataset Images: 28
- Test Dataset Images: 5
- Weights are saved for every 10th epoch
- Best weight is saved separately
- Accuracy metrics:
  - Mean Average Precision (mAP) = 91.9%
  - Box Detection Accuracy: 89.1%
  - Mask Segmentation Accuracy: 89.9%

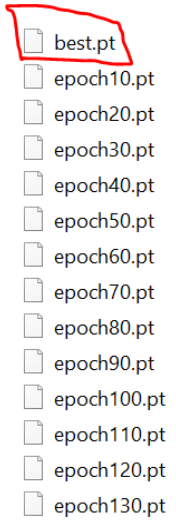
```
1  model.train(data="config.yaml", epochs=200, imgsz=640, batch=5, save_period = 10)  # train the model
```

```
      Epoch    GPU_mem   box_loss   seg_loss   cls_loss   dfl_loss  Instances       Size
  0%|          | 0/7 [00:00<?, ?it/s]C:\ProgramData\Anaconda3\lib\site-packages\torch\amp\autocast_mode.py:198: UserWarning:
User provided device_type of 'cuda', but CUDA is not available. Disabling
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
      1/200         0G      1.151       1.01      1.334      1.105         47        640: 100%|██████████| 7/7 [01:20<0
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP
                   all          6         77      0.815      0.685      0.727      0.501      0.815      0.685      0.727
0.432

      Epoch    GPU_mem   box_loss   seg_loss   cls_loss   dfl_loss  Instances       Size

      2/200         0G      1.163       1.013      1.342      1.179         49        640: 100%|██████████| 7/7 [01:10<0
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP
                   all          6         77      0.819      0.675      0.732      0.513      0.819      0.675      0.732
0.429

      Epoch    GPU_mem   box_loss   seg_loss   cls_loss   dfl_loss  Instances       Size
      3/200         0G      1.212       1.072      1.414      1.155         56        640: 100%|██████████| 7/7 [01:06<0
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP
                   all          6         77      0.777      0.675      0.722      0.496      0.777      0.675      0.722
0.433

      Epoch    GPU_mem   box_loss   seg_loss   cls_loss   dfl_loss  Instances       Size
      4/200         0G      1.216       1.059      1.293      1.114         63        640: 100%|██████████| 7/7 [01:07<0
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95)     Mask(P          R      mAP
                   all          6         77      0.811      0.688      0.737        0.5      0.811      0.688      0.737
0.464
```

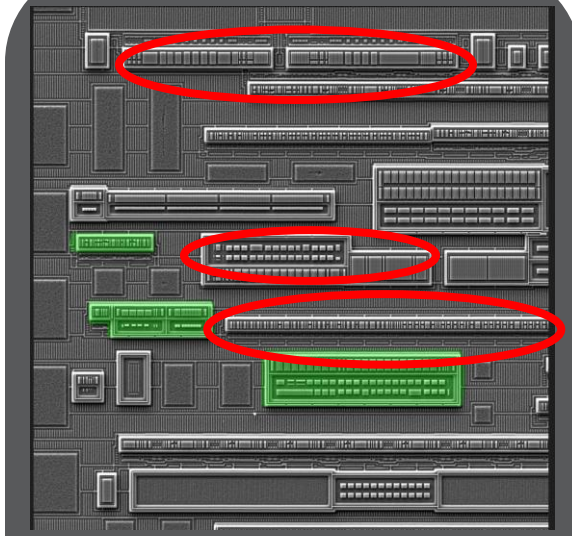








Model Training



Epoch 10

- **Bounding Box Loss:** 1.282
- **Segmentation Loss:** 1.139
- **Classification Loss:** 1.302

Epoch 50

- **Bounding Box Loss:** 1.251
- **Segmentation Loss:** 0.8734
- **Classification Loss:** 1.177

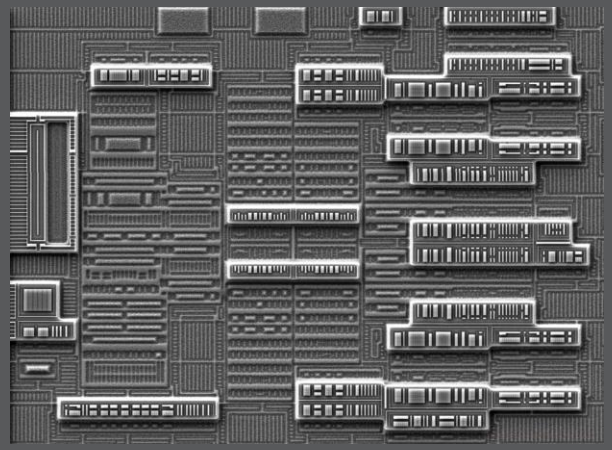
Epoch 100

- **Bounding Box Loss:** 1.101
- **Segmentation Loss:** 0.9855
- **Classification Loss:** 1.094

Epoch 200

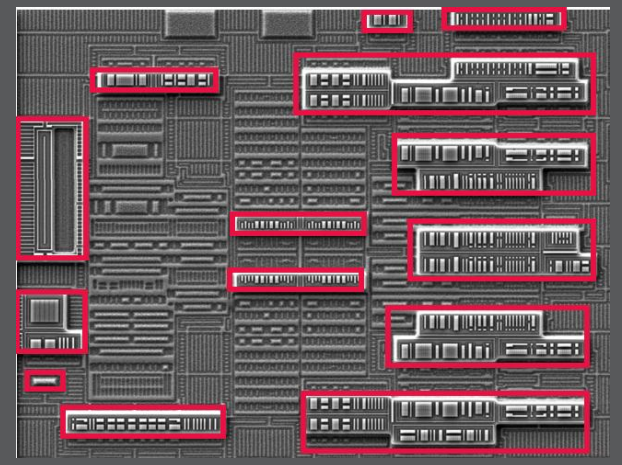
- **Bounding Box Loss:** 1.008
- **Segmentation Loss:** 0.9433
- **Classification Loss:** 1.073

Trench Viz Testing

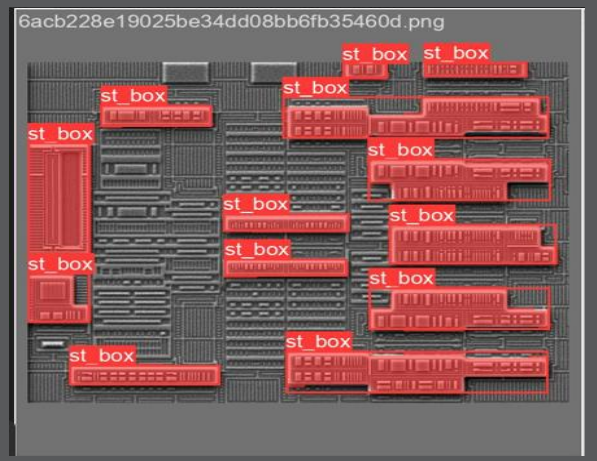


Raw Image passed to YOLO for segmentation

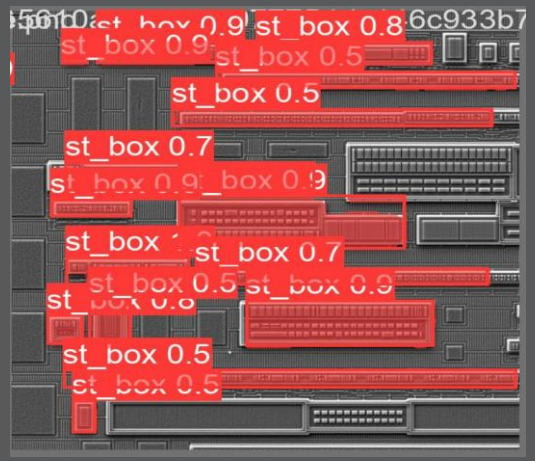
Feature Map extraction: Extracts high level features and make feature maps for the activation functions.



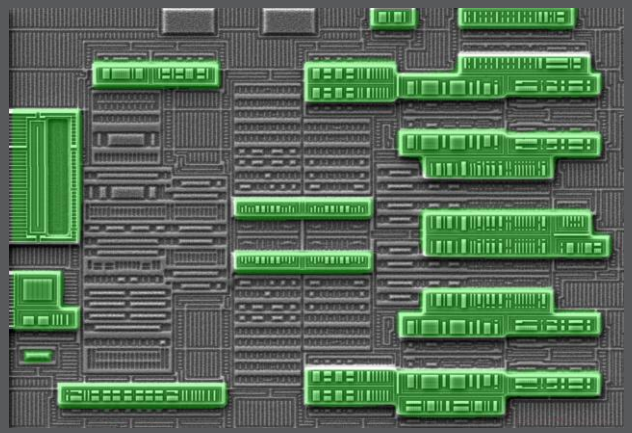
YOLO object detection



YOLO Object/ Instance Segmentation

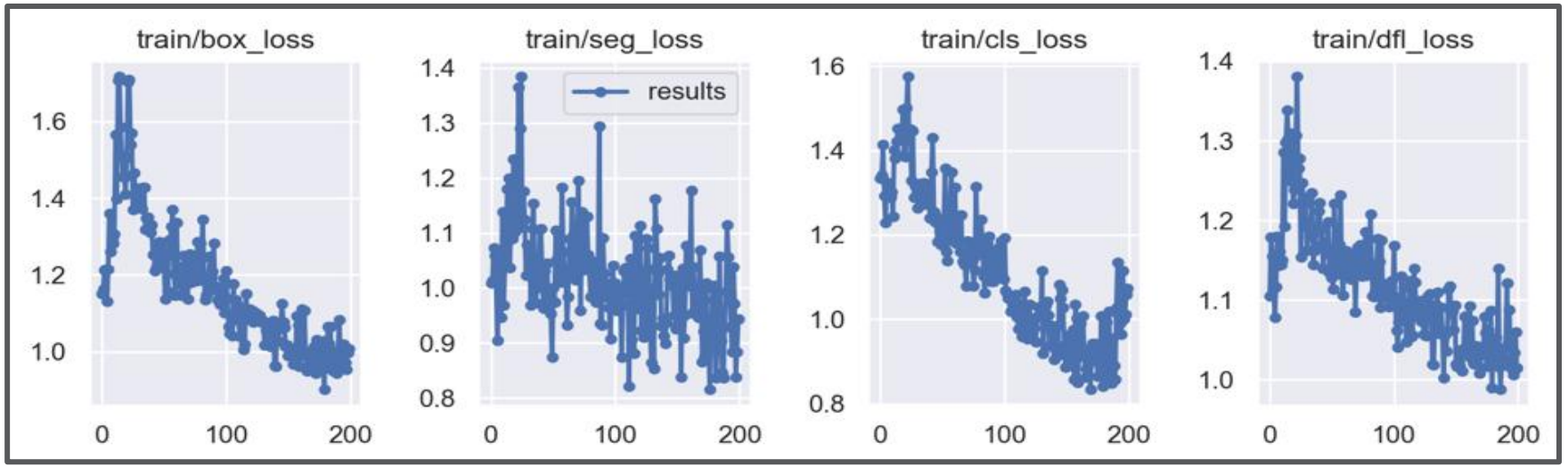


YOLO Object Classification and probability prediction



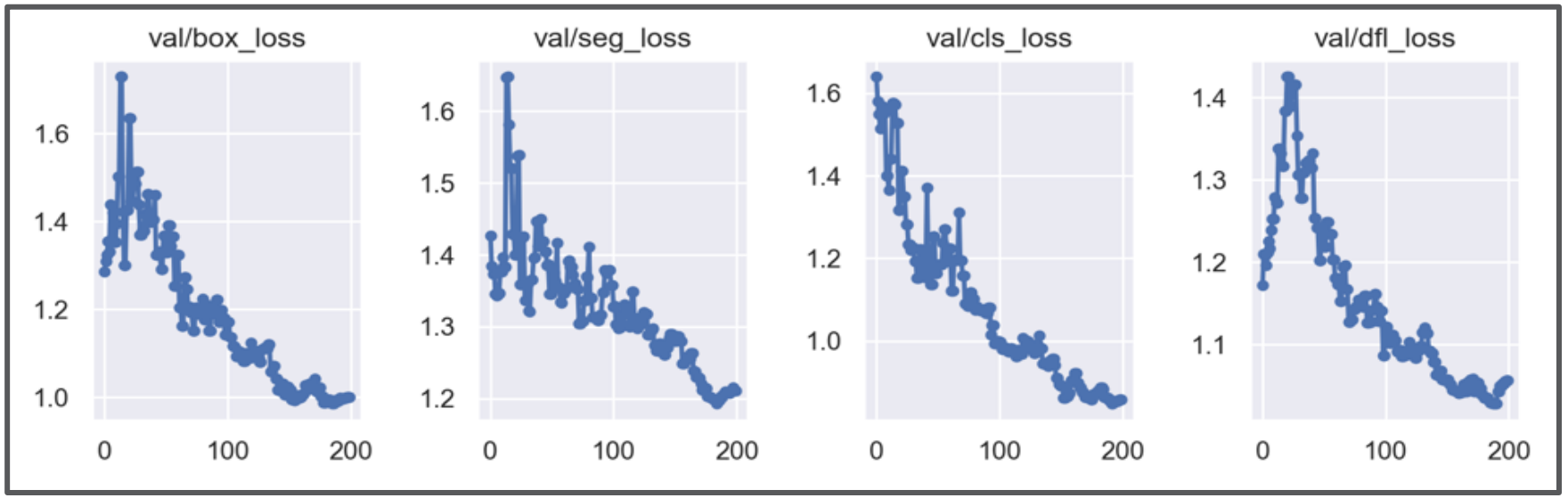
IOU for final segmentation masks

Training Validation Loss Plots



These plots are drawn for the **training data**
From Left to Right plots:

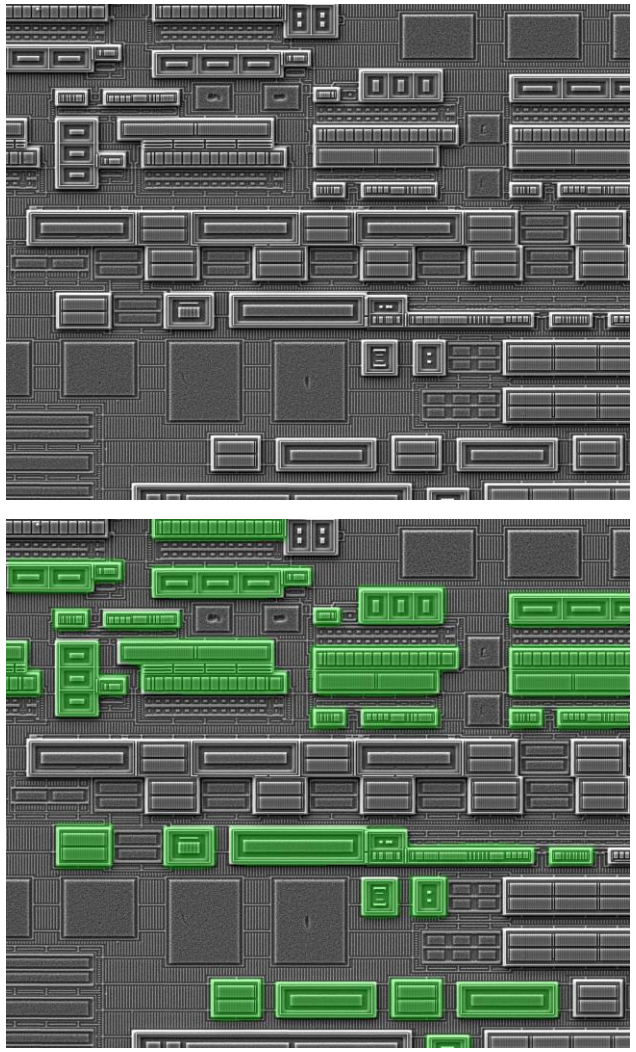
(a) Loss curve for Bounding Box detection
(b) Loss curve for Segmentation
(c) Loss curve for Classification
(d) Dual Focal Loss (DFL)



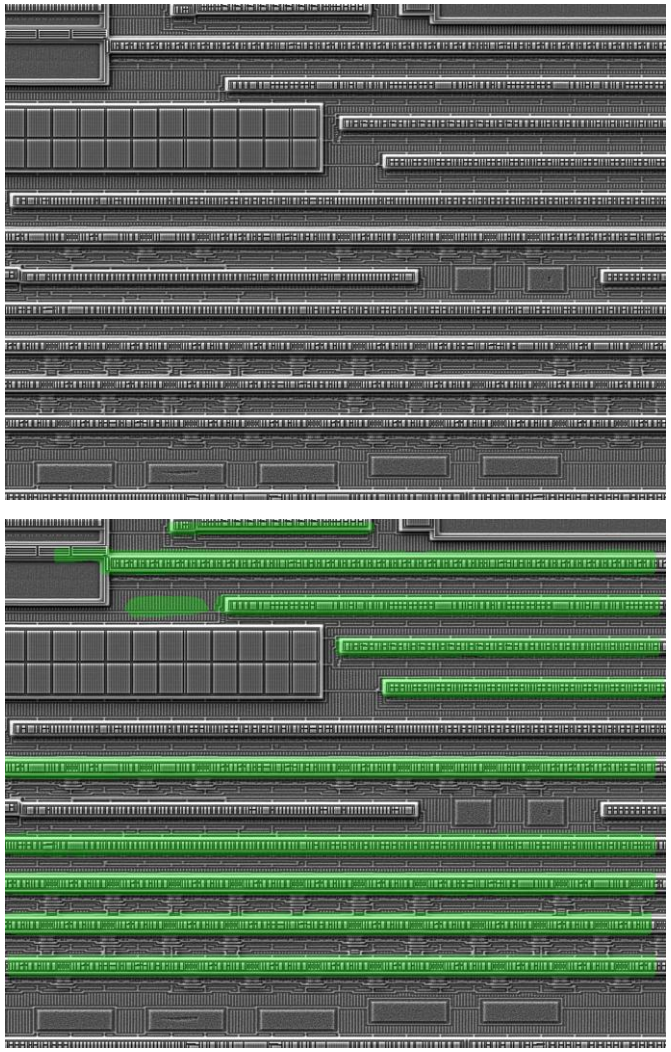
These plots are drawn for the **validation data**
From Left to Right plots:

(a) Loss curve for Bounding Box detection
(b) Loss curve for Segmentation
(c) Loss curve for Classification
(d) Dual Focal Loss (DFL)

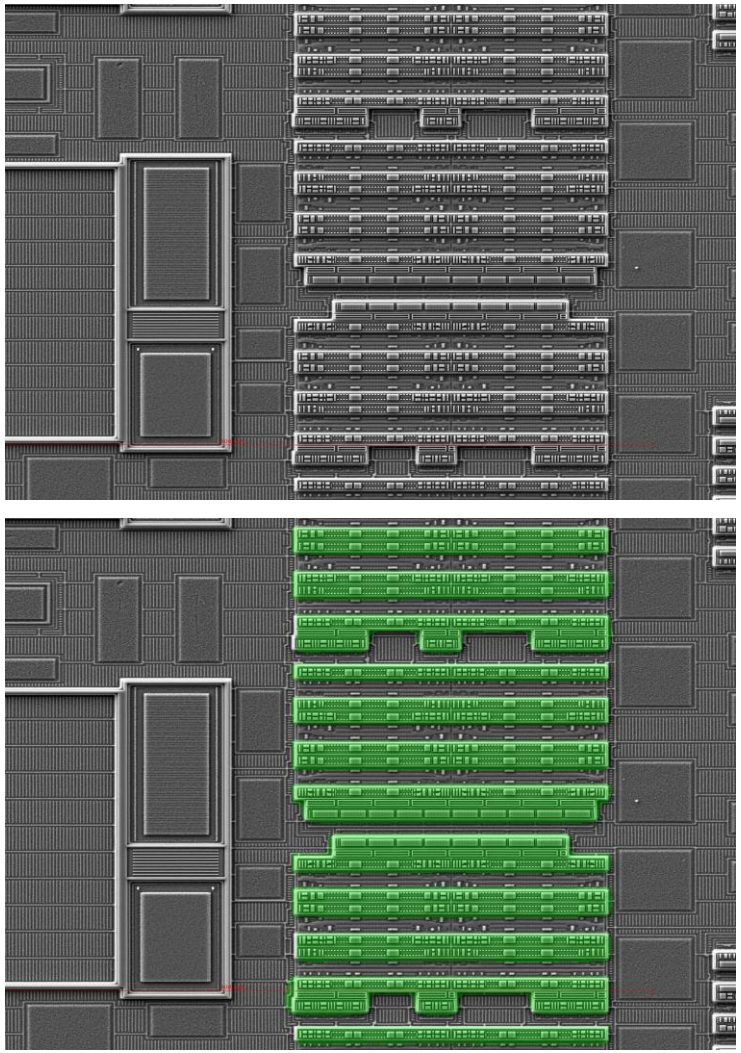
More results



Segmentation Accuracy: 92%
Time: 0.1ms

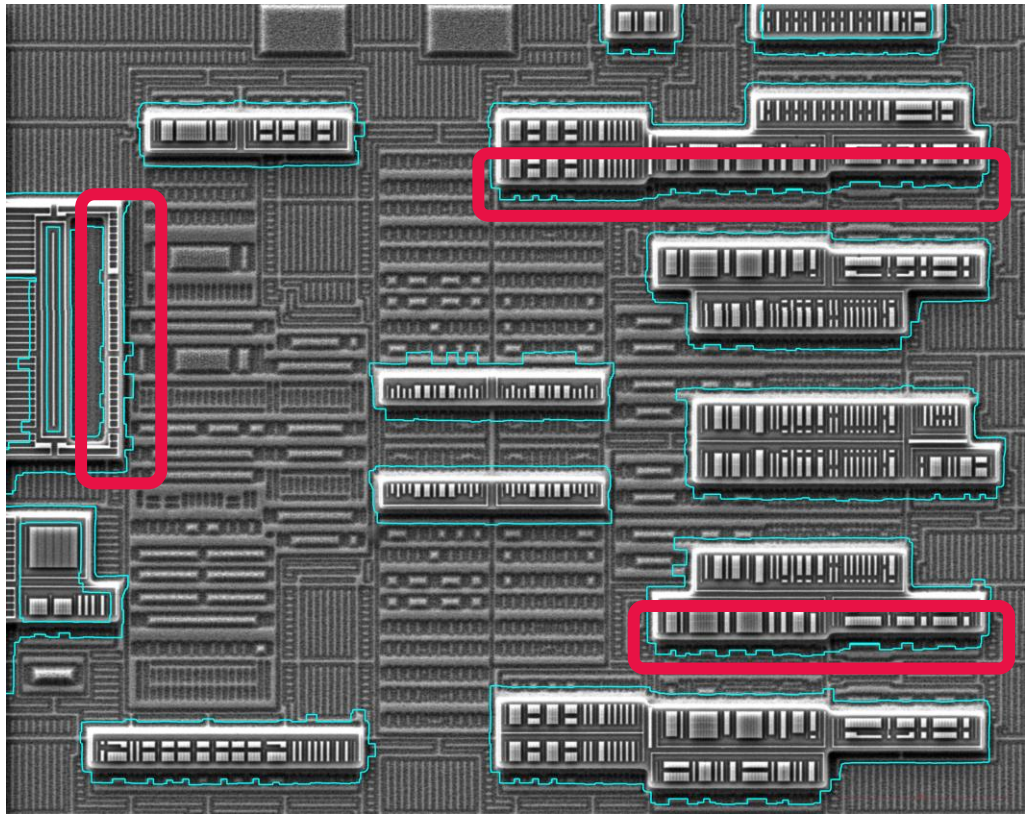


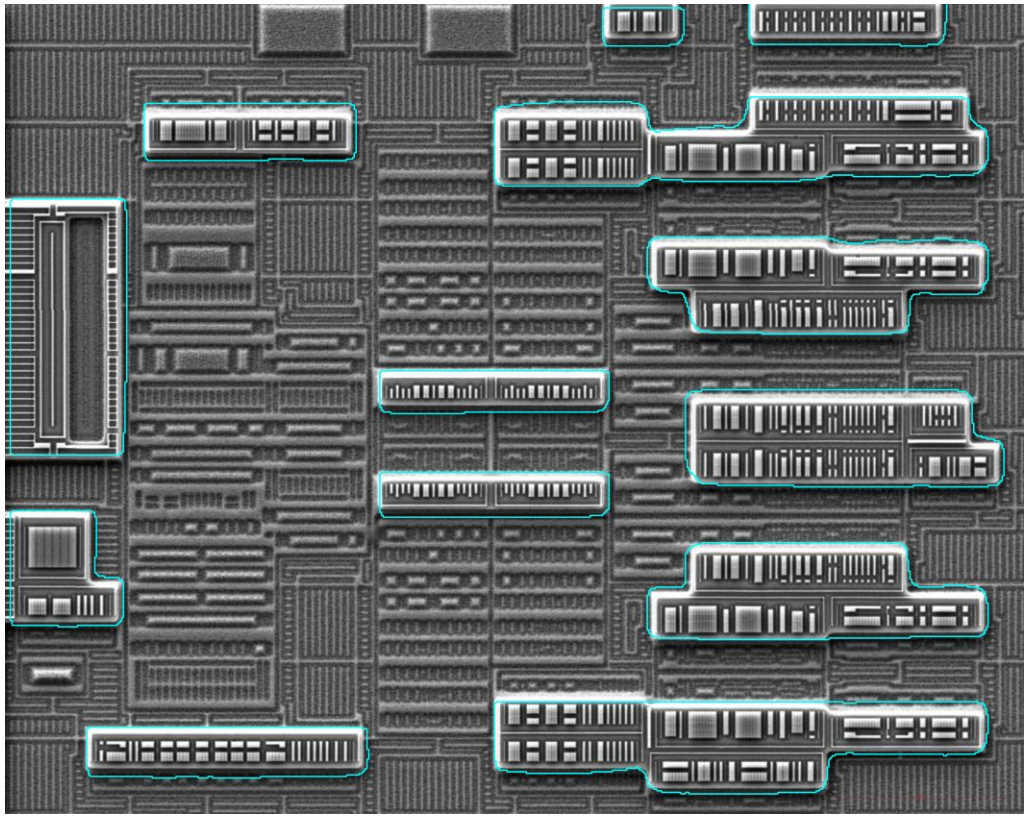
Segmentation Accuracy: 95%
Time: 0.1ms



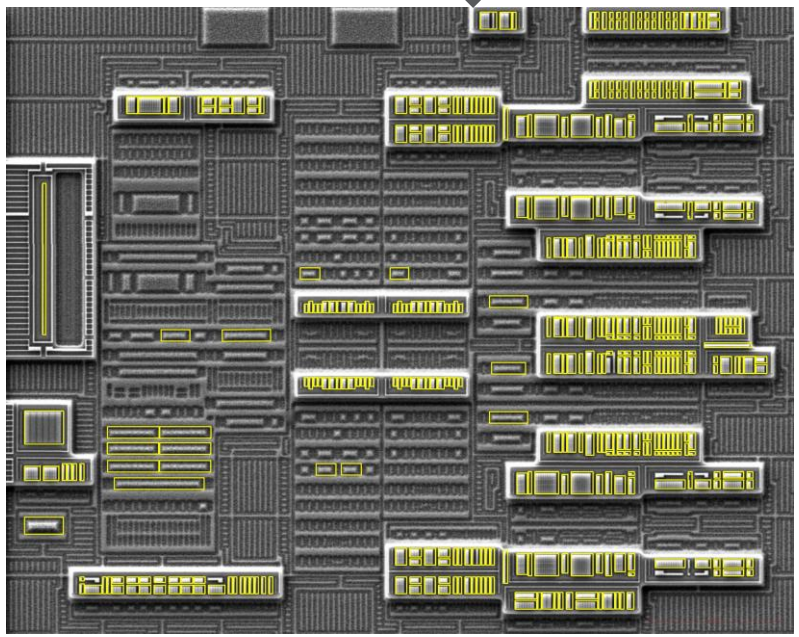
Segmentation Accuracy: 98%
Time: 0.1ms

Current V/S New state

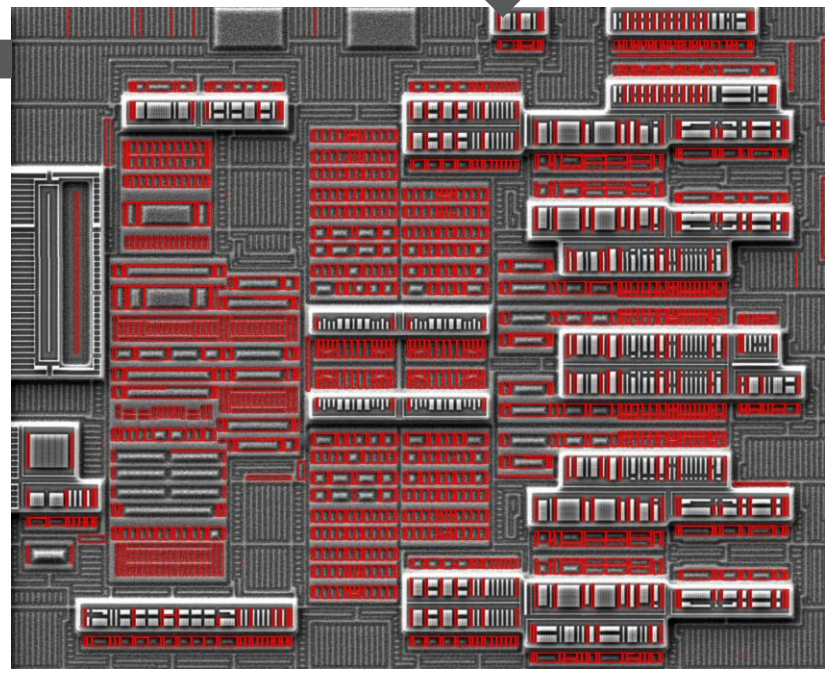




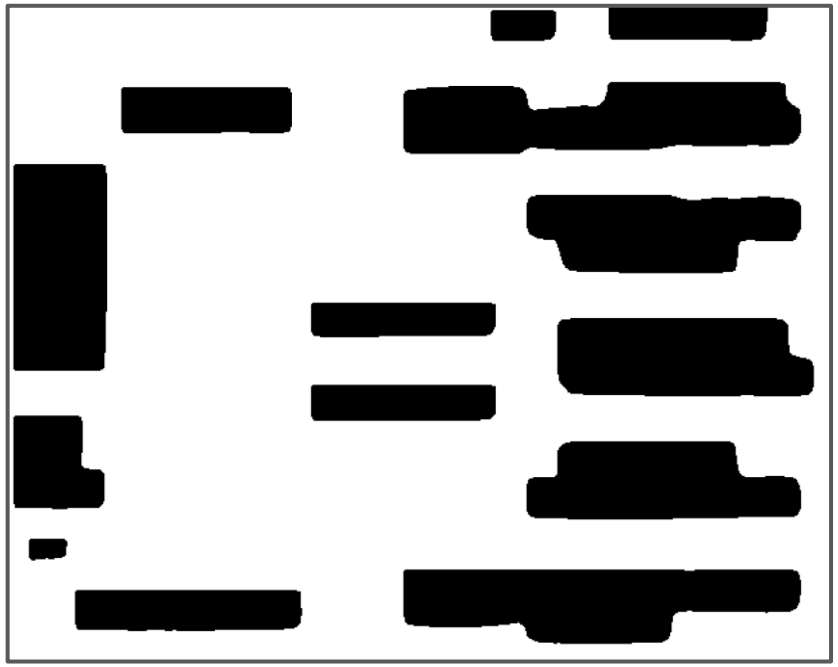
Overlay on ST layer



Step9: This is all N-Wells.
Parameter: Where Intensity > 160



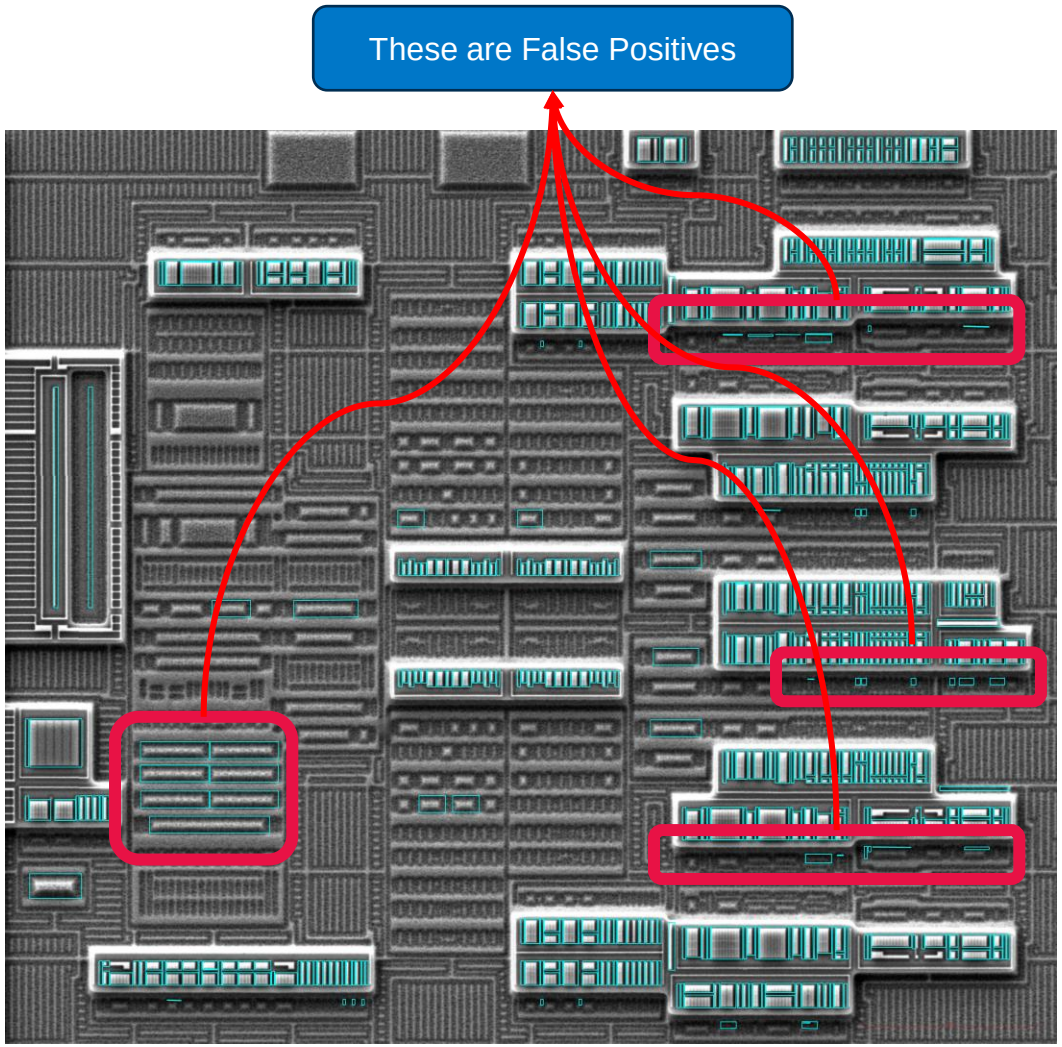
Step8: This is all P-Wells.
Parameter: Where Intensity < 160



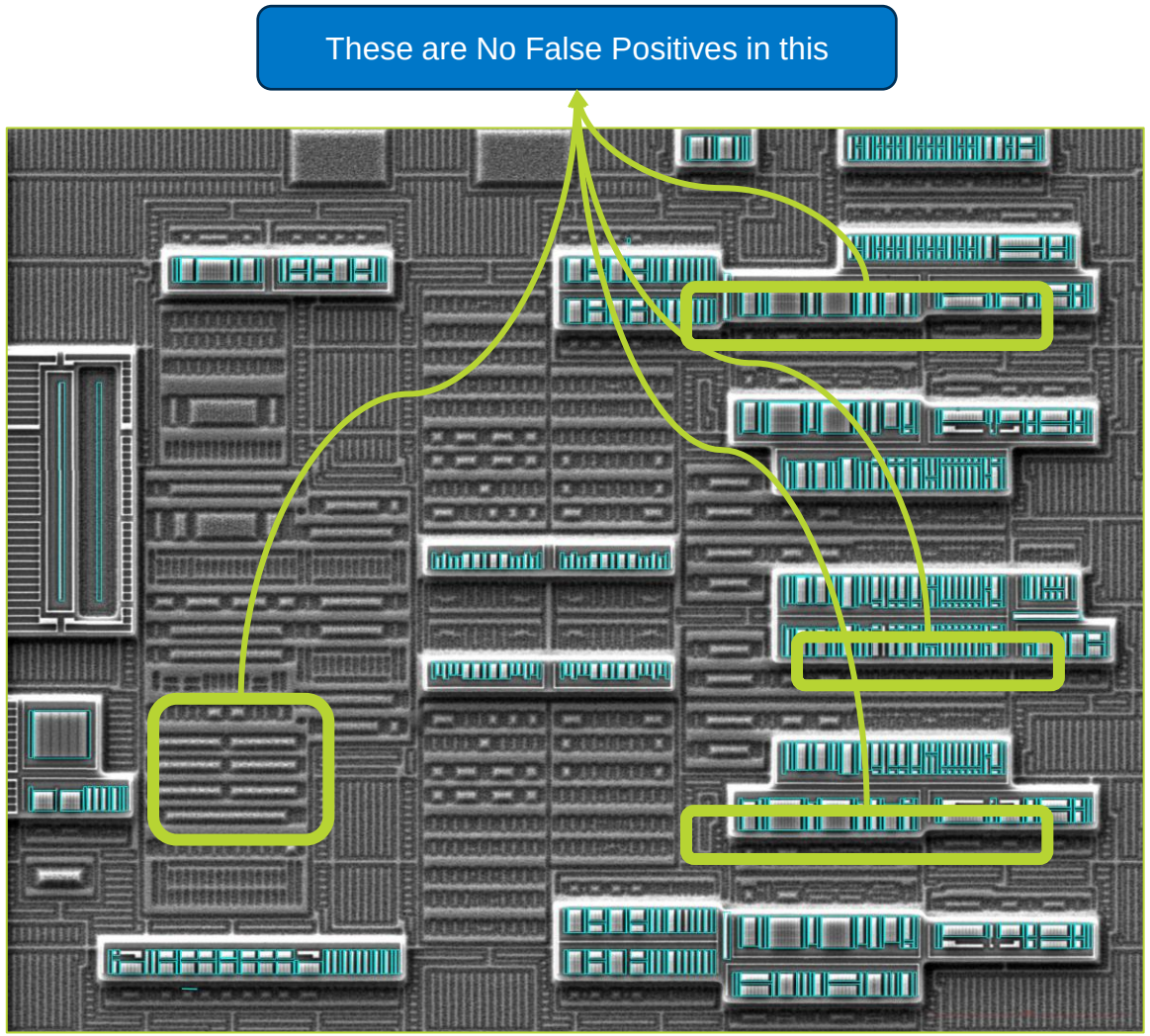
Step7: Overlay the detections on ST layer and filter basis intensities.
Parameter: 10% Pixels should be <20

Micron Highly Confidential

N-Well detection Improvements



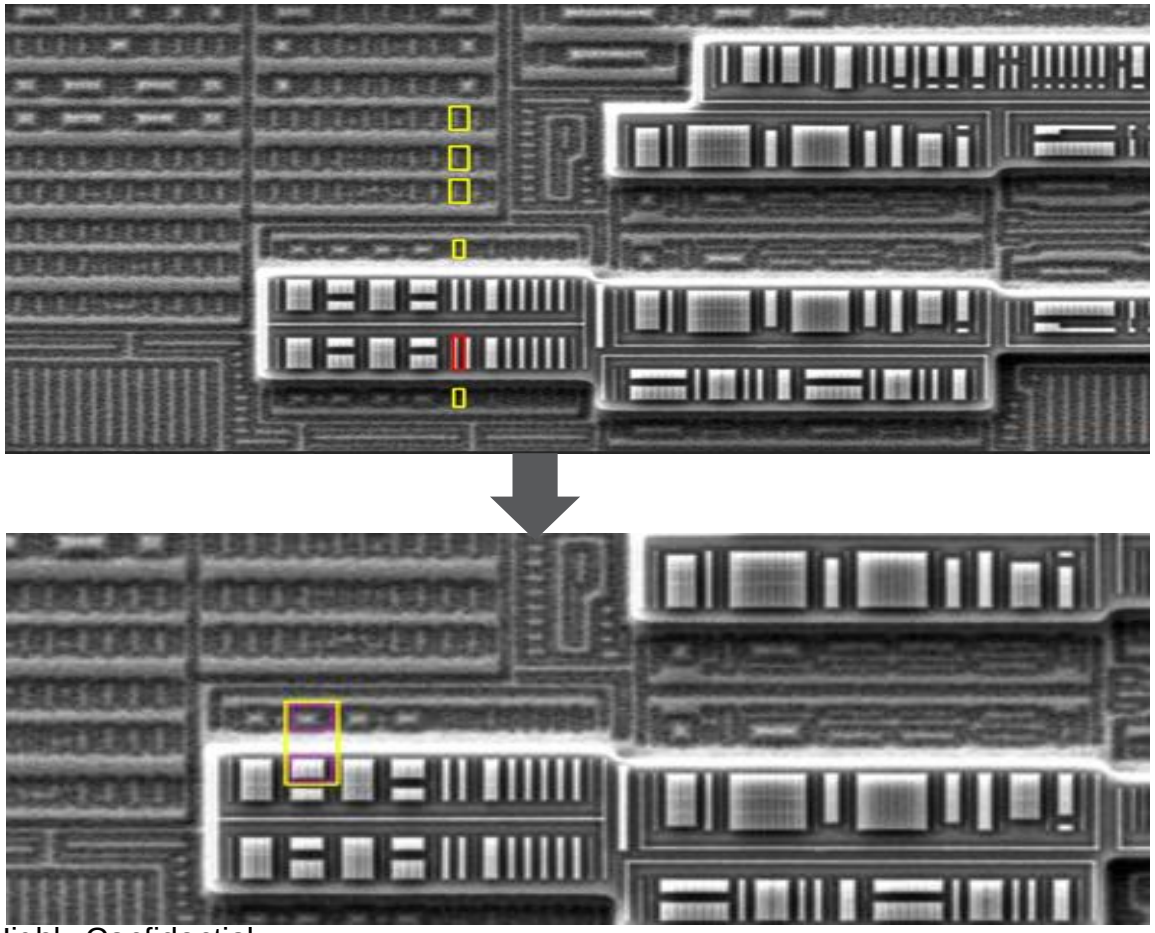
Total Number of N-wells detected: 256
Correct N-wells detected: 211
Falsely detected N-wells: **45**

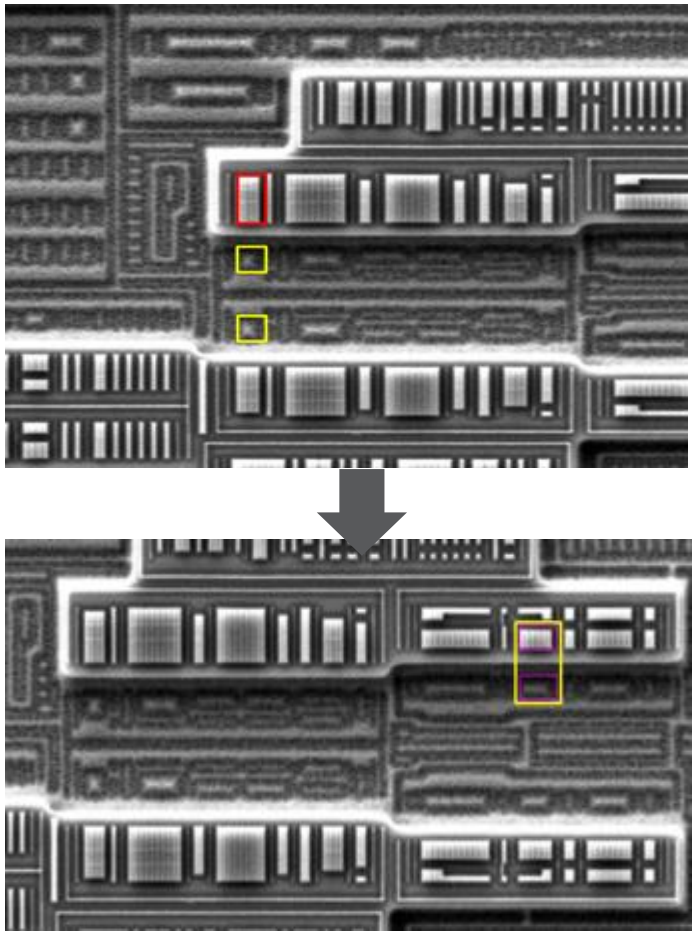


Total Number of N-wells detected: 211
Correct N-wells detected: 211
Falsely detected N-wells: **0**

3-Fold Approach to Detect all the Standard Cells

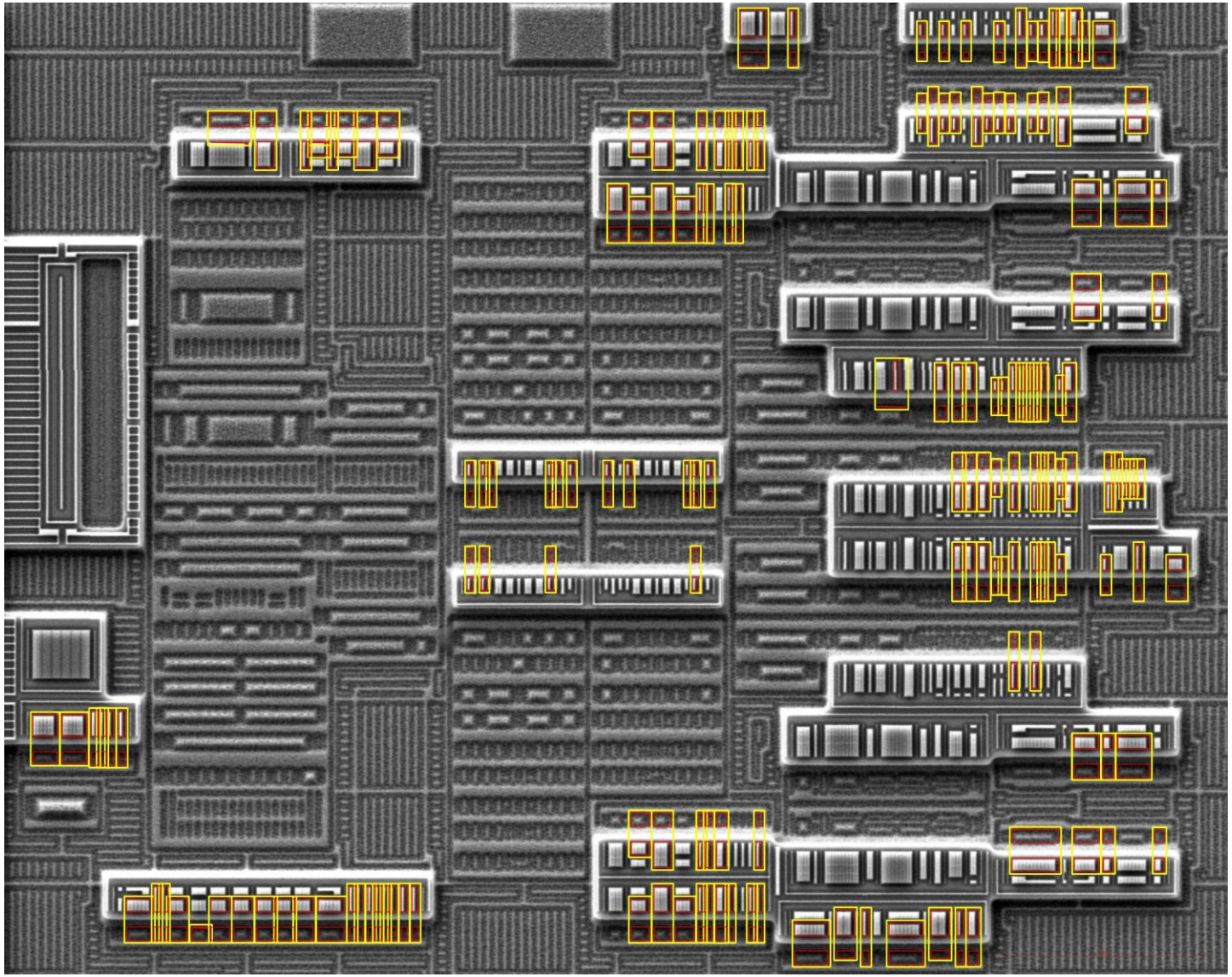
- For a corresponding N-well, check which are the P-wells which have same center point.
- Check If they have same width
- Choose the one which is nearest (make pair of N-well and P-well).





Standard Cells Detected

- Number of N-wells detected: 211
- Number of P-wells detected: 456
- Number of Standard cells detected: 142



Invention: Benefits

- **What are the technical advantages?**
  - Usage of new and innovative methods improves the component and StdCell boundary detections significantly
  - In-house trained models on custom dataset (starting from data collection, preparation and to prediction)
  - Combination of classical CV approaches + New SOTA models
  - Standardized approach significantly reduce the manual efforts required on the next generation competitor’s reverse engineering projects
  - Enables the platform to train on Micron’s patents in FEOL area and find the match of similar implementations in competitors' products to detect IP infringement
- **What are the strategic advantages?**
  - Reduces component detection time by 98% (from ~2.5-man months to ~1-day)
  - With this approach, we can explore >10% of area of competitors products
  - Develop broader understanding of the competitors design rules
  - Understand industry leading architectures and circuit trends in significantly shorter duration
  - Creating the scalable database of competitor’s FEOL components which will be used in the next-phases of the StdCell/Circuit identification
  - Integrating competitor’s innovation in Micron’s product by shifting left and intercepting DBR leads to faster time to market
  - Parallelism of this approach, allows to reverse engineer more than one competitor’s part at any given time

Invention: Information

- Invention Conception Date: 02/02/2024
- First Named (Primary) Inventor: Jimit Shah
- Co-inventors: Prasun Agarwal, Santosh Paudel


Micron®